

# CPJ-Cobrança AI

## Manual Técnico & Documentação de Arquitetura

### Desafio Técnico Dev Pleno

Solução de Agente de IA para Apoio de Processo de Desenvolvimento

**Autor:** Cledson Santos

**Data:** Maio de 2026

**Versão:** 1.0.0

# Introducao

---

## Objetivo deste capitulo

---

Este capitulo apresenta o projeto desenvolvido para o case tecnico CPJ-Cobrança AI em nivel executivo e tecnico. A ideia e deixar claro, desde o inicio, que esta documentacao faz parte da entrega do case: ela explica o contexto do desafio, a proposta da solucao, o que foi construido e por que as decisoes tecnicas foram tomadas.

O objetivo e permitir que um avaliador entenda rapidamente qual problema foi resolvido, qual solucao foi entregue, quais diferenciais tecnicos existem e por onde continuar a leitura.

Esta introducao nao tenta explicar todos os detalhes internos. Arquitetura, contratos de API, persistencia, deploy, testes e extensibilidade serao detalhados nos proximos capitulos.

## Contexto do case

---

O projeto nasceu como resposta a um case tecnico com foco em cobranca, automacao e uso pratico de IA no ciclo de desenvolvimento. Em vez de entregar apenas um prototipo pontual, a proposta foi construir uma base de produto avaliavel: API, persistencia, agentes, painel web, documentacao interativa, Docker, testes e caminhos claros de evolucao.

Nesse contexto, o CPJ-Cobrança AI e uma solucao para apoiar atividades tecnicas relacionadas a analise de codigo, aderencia a requisitos, documentacao, geracao de testes e revisao de Pull Requests com auxilio de agentes de IA.

O nome do projeto reflete o objetivo do case: criar uma ferramenta aplicavel ao ambiente CPJ-Cobrança, onde revisoes, documentacao, verificacao de regras e qualidade tecnica podem ser aceleradas por fluxos automatizados, mas ainda mantendo rastreabilidade para avaliacao humana.

Em vez de tratar a IA como uma chamada isolada de chat, o projeto organiza cada capacidade em fluxos especializados. Esses fluxos combinam validacao de contratos, ferramentas deterministicas, prompts versionados, modelos configuraveis, grafos LangGraph, persistencia de historico, telemetria e uma interface web para acompanhamento operacional.

O resultado e uma API pronta para avaliacao tecnica, acompanhada por um painel administrativo Next.js e por uma estrutura de execucao local ou containerizada.

## O que o case pedia

---

O enunciado do case solicitava a construção de um microserviço backend containerizado, com API REST, capaz de expor um agente de IA para automatizar quatro fluxos do processo de desenvolvimento do CPJ-Cobrança.

Os quatro fluxos obrigatórios eram:

- **Code Review Automatizado:** endpoint `POST /api/v1/review`, recebendo um trecho de código, linguagem e contexto opcional, e retornando qualidade geral, nota, problemas encontrados, pontos positivos e resumo.
- **Avaliação de Aderência a Tarefa:** endpoint `POST /api/v1/compliance`, cruzando uma descrição de tarefa com o código implementado para apontar requisitos atendidos, ausentes e parcialmente atendidos.
- **Geração de Documentação Técnica:** endpoint `POST /api/v1/document`, gerando documentação técnica ou operacional a partir de código informado.
- **Geração de Testes Unitários:** endpoint `POST /api/v1/tests`, produzindo casos de teste, dicas de cobertura e o conteúdo completo de um arquivo de teste.

Também eram obrigatórios endpoints auxiliares para operação e rastreabilidade:

- `GET /health`, para healthcheck da aplicação;
- `GET /api/v1/history`, para listar as últimas execuções;
- `GET /api/v1/history/{id}`, para consultar o resultado completo de uma execução específica.

Do ponto de vista técnico, o case exigia:

- persistência das execuções do agente em banco de dados;
- integração com algum provedor LLM, documentando modelo, configuração e custo estimado quando aplicável;
- Dockerfile, `docker-compose.yml` e `.env.example`;
- subida da aplicação com `docker compose up --build`;
- documentação clara de setup, variáveis de ambiente e exemplos funcionais;
- repositório público com código-fonte, configuração Docker e coleção de exemplos via Postman, Insomnia ou arquivo `.http`.

O README também era parte central da entrega. O enunciado pedia que ele explicasse decisões técnicas, instruções de execução com Docker, configuração de ambiente, exemplos completos de request e response para cada fluxo, diferenciais implementados e o que seria feito com mais tempo.

## Critérios de avaliação do case

O case definia avaliação de 1 a 5 em cada critério, com pesos diferentes:

| Critério                   | Peso | O que era avaliado                                                              |
|----------------------------|------|---------------------------------------------------------------------------------|
| Funcionalidade             | 25%  | Quatro fluxos funcionando, Docker subindo e endpoints respondendo corretamente. |
| Qualidade do agente / IA   | 25%  | Qualidade dos prompts, utilidade real das saídas e tratamento de falhas do LLM. |
| Qualidade do código        | 20%  | Organização, legibilidade, separação de responsabilidades e consistência.       |
| README e documentação      | 15%  | Clareza, decisões justificadas e exemplos que realmente funcionam.              |
| Persistência e modelagem   | 10%  | Schema adequado, queries corretas e dados consistentes.                         |
| Diferenciais implementados | 5%   | Qualidade e utilidade dos itens extras escolhidos.                              |

O enunciado deixava claro que uma solução simples, legível e funcional teria mais valor do que uma arquitetura sofisticada que não subisse ou que não tivesse instruções de uso confiáveis.

Também valorizava a forma de pensar, documentar, comunicar trade-offs e construir algo realista com IA.

Entre os diferenciais sugeridos estavam LangGraph ou LangChain, prompt templating estruturado, cache por hash, logs estruturados, tracing com LangSmith, batch, streaming via Server-Sent Events, webhook callback, testes automatizados e validação de schema na entrada e na saída.

## Problema resolvido

---

Atividades como revisar código, verificar se uma entrega atende uma tarefa, gerar documentação ou sugerir testes costumam depender de leitura manual, critérios pouco padronizados e conhecimento disperso entre pessoas do time.

O projeto resolve esse problema criando uma camada automatizada e rastreável para apoiar essas decisões. A API recebe código, contexto ou dados de Pull Request, executa um fluxo especializado e devolve uma resposta estruturada que pode ser armazenada, revisada, auditada e comparada depois.

Essa abordagem ajuda em quatro pontos principais:

- padronizar análises técnicas com contratos previsíveis;
- reduzir trabalho repetitivo de revisão, documentação e testes;
- registrar histórico, steps, modelo usado, tokens e custos;
- permitir evolução controlada de prompts, modelos e agentes.

## Visão geral da solução

---

A solução é composta por duas partes principais:

- **API backend:** aplicação Node.js com TypeScript e Fastify, responsável por expor endpoints HTTP, validar payloads, executar fluxos de IA, persistir histórico e disponibilizar documentação OpenAPI.
- **Painel web admin:** aplicação Next.js em [apps/web](#), usada para acompanhar histórico, prompts, modelos, status da API, custos, tokens e execução guiada dos fluxos.

O backend usa PostgreSQL via Prisma para persistir execucoes, steps, telemetria, prompts versionados, catalogo de modelos e resumos de batch. Os fluxos de IA usam LangGraph.js e LangChain.js com OpenRouter como provedor LLM. LangSmith pode ser ativado opcionalmente para tracing.

## Capacidades entregues

---

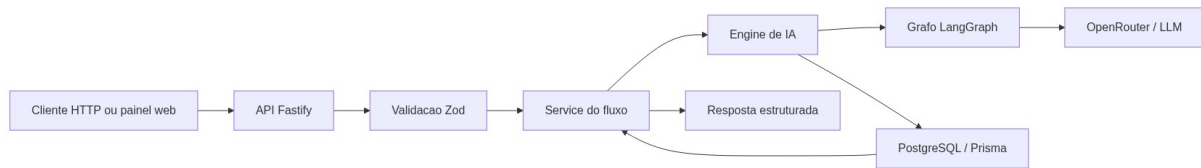
O projeto entrega os seguintes fluxos e recursos:

- **review**: revisao de codigo multi-linguagem para TypeScript, JavaScript, Python e PHP.
- **review/stream**: revisao de codigo com Server-Sent Events para acompanhar a execucao em tempo real.
- **review/pull-request**: revisao de Pull Request no GitHub, com padroes de codigo, seguranca, consistencia de projeto e criterios Jira opcionais.
- **compliance**: avaliacao de aderencia entre descricao da tarefa e codigo implementado.
- **document**: geracao de documentacao tecnica ou operacional a partir de codigo.
- **tests**: geracao de estrategia, casos e arquivo de testes.
- **tests/pull-request**: geracao de testes unitarios baseada nas funcoes e trechos criticos alterados em um Pull Request.
- **batch**: execucao sequencial de multiplos fluxos em uma unica requisicao.
- **history**: consulta das execucoes persistidas, incluindo detalhes e steps.
- **analytics**: visao agregada de uso, tokens e custos.
- **prompts**: cadastro, consulta e ativacao de versoes de prompt por fluxo.
- **models**: cadastro, ativacao e selecao de modelo padrao global.
- **webhook**: callback opcional ao finalizar fluxos principais.
- **OpenAPI/Swagger**: documentacao interativa em **/docs**.

## Como a solucao funciona em alto nivel

---

O fluxo padrao segue esta sequencia conceitual:



Em termos praticos, uma request entra pela API, e validada pelos contratos compartilhados, passa pelo service do modulo correspondente e chega ao engine do fluxo. O engine resolve prompt e modelo, verifica cache quando aplicavel, executa o grafo LangGraph, persiste o resultado, registra telemetria e retorna um JSON estruturado.

## Principais diferenciais tecnicos

O case vai alem de um endpoint simples chamando um modelo de linguagem. Os principais diferenciais sao:

- **Arquitetura modular:** cada fluxo possui controllers, routes, services, engines, graphs, agents, prompts e tools separados por responsabilidade.
- **Contratos estruturados:** entradas e saidas usam schemas compartilhados, reduzindo ambiguidade entre API, agentes, testes e painel web.
- **Grafos de agentes:** LangGraph organiza etapas observaveis, incluindo roteamento por linguagem, tools deterministicas, agentes especialistas e agregacao.
- **Persistencia completa:** historico, steps, telemetria, custos, tokens, cache e batch ficam registrados em PostgreSQL.
- **Governanca de IA:** prompts e modelos nao ficam apenas hardcoded; podem ser versionados, ativados e sobrescritos por request.
- **Operacao local e containerizada:** Docker Compose sobe API, banco e painel web, aplicando migrations e seed antes do start da API.
- **Avaliabilidade:** a solucao tem testes automatizados, OpenAPI, exemplos HTTP e comandos de verificacao para facilitar revisao tecnica.

## Leitura rapida para avaliacao

Para avaliar o projeto rapidamente, a ordem recomendada e:

1. Ler este capitulo para entender o contexto e os diferenciais.
2. Abrir `02-escope-funcional.md` para ver tudo que foi entregue.

3. Ler `04-arquitetura.md` e `05-fluxos-de-ia-e-agentes.md` para entender as decisoes centrais.
4. Usar `10-como-rodar.md` para executar localmente.
5. Consultar `12-testes-validacao.md` para validar a entrega.

Enquanto os capitulos seguintes ainda nao forem escritos, o README raiz e a documentacao OpenAPI em `/docs` continuam sendo as referencias praticas para execucao e contratos.

## Proximo capitulo

---

O proximo documento planejado e `02-escopo-funcional.md`, que detalhara o que cada fluxo faz, quais recursos estao implementados e quais responsabilidades ficam fora do escopo atual.



# Escopo Funcional

---

## Objetivo deste capítulo

---

Este capítulo descreve o que a solução entrega do ponto de vista funcional. Aqui o foco não é explicar a arquitetura interna, mas deixar claro quais fluxos existem, para que servem, como se relacionam com o case técnico e quais recursos de apoio foram implementados ao redor deles.

## Visão geral do escopo

---

O CPJ-Cobrança AI foi construído como uma plataforma de apoio ao processo de desenvolvimento. A entrega principal é uma API REST com agentes de IA capazes de analisar artefatos técnicos e retornar respostas estruturadas, rastreáveis e persistentes.

O escopo funcional está organizado em quatro grupos:

- **Fluxos obrigatórios do case:** review, compliance, document e tests.
- **Diferenciais de automação:** batch, streaming SSE, webhooks, review de Pull Request e geração de testes por Pull Request.
- **Governança e rastreabilidade:** histórico, analytics, prompts versionados, catálogo de modelos, cache, telemetria e persistência.
- **Experiência operacional:** Swagger/OpenAPI, arquivo `.http`, Docker Compose e painel web admin.

## Leitura do escopo na perspectiva do case

---

Para um avaliador técnico, este capítulo deve ser lido em três camadas:

- **o que era obrigatório no enunciado** e precisa existir para o case estar funcional;
- **o que amplia a entrega além do mínimo pedido**, aumentando o valor técnico e a proximidade com um cenário real de uso;
- **o que ainda não faz parte desta versão**, mas aparece como evolução natural para um produto mais maduro.

Na pratica, os fluxos `review`, `compliance`, `document` e `tests`, junto com `GET /health` e os endpoints de historico, representam o nucleo obrigatorio do case. Os demais recursos mostram como a solucao foi estendida para melhorar operacao, avaliacao, governanca e uso no dia a dia.

## Fluxos obrigatorios do case

---

Esta secao descreve o coracao funcional pedido no enunciado. Sem esses fluxos, o case nao estaria completo do ponto de vista da entrega minima esperada.

### Code Review Automatizado

Endpoint principal:

- `POST /api/v1/review`

Este fluxo recebe um trecho de codigo, a linguagem e um contexto opcional. A resposta apresenta uma avaliacao estruturada com qualidade geral, nota, problemas encontrados, sugestoes, pontos positivos e resumo.

No case, esse fluxo deveria verificar pontos como clareza de nomenclatura, tratamento de erros, vazamentos de recurso, complexidade percebida e padroes comuns de seguranca. A implementacao amplia esse escopo com:

- suporte a TypeScript, JavaScript, Python e PHP;
- roteamento por linguagem;
- tools deterministicas antes das chamadas ao LLM;
- agentes especialistas para diferentes criterios de revisao;
- agregacao final da analise;
- persistencia de execucao, steps e telemetria;
- cache por hash para evitar reprocessar entradas identicas.

### Avaliacao de Aderencia a Tarefa

Endpoint principal:

- `POST /api/v1/compliance`

Este fluxo recebe uma descrição de tarefa, o código implementado e a linguagem. A resposta informa se a implementação está aderente, calcula um score de conformidade e separa requisitos cobertos, ausentes e parcialmente atendidos.

O objetivo funcional é cruzar o que a tarefa pedia com o que o código realmente faz. Isso ajuda a identificar lacunas de implementação, comportamentos parciais, requisitos ignorados e pontos que exigem revisão humana.

## Geracao de Documentacao Tecnica ou Operacional

Endpoint principal:

- `POST /api/v1/document`

Este fluxo recebe código, linguagem e tipo de documentação. O tipo pode orientar uma documentação mais técnica, voltada a desenvolvedores, ou mais operacional, voltada a squads, produto ou pessoas que precisam entender o comportamento sem entrar nos detalhes internos do código.

A resposta estrutura título, descrição, entradas, saídas, efeitos colaterais, exemplo de uso e observações. O foco é transformar código em documentação útil, revisável e reaproveitável.

## Geracao de Testes Unitarios

Endpoint principal:

- `POST /api/v1/tests`

Este fluxo recebe código, linguagem e framework de teste desejado. A resposta retorna o framework, um arquivo de teste completo, uma lista de casos cobertos e dicas de cobertura para revisão manual.

O objetivo é acelerar a criação de testes para caminhos felizes, casos de borda e situações de erro, sem substituir a revisão técnica de quem conhece o domínio do módulo.

## Diferenciais implementados

---

Os itens abaixo não eram obrigatórios no enunciado, mas foram implementados para tornar a solução mais completa, mais demonstrável e mais próxima de um uso real em ambiente técnico.

## Streaming do review

Endpoint:

- `POST /api/v1/review/stream`

Este endpoint executa o mesmo fluxo de review, mas responde via Server-Sent Events. Ele permite acompanhar eventos como inicio, steps intermediarios, resultado, erro e finalizacao.

Esse recurso foi implementado como diferencial de experiencia de uso para processamentos que podem demorar mais do que uma request simples.

## Execucao em lote

Endpoint:

- `POST /api/v1/batch`

O batch permite enviar varios itens em uma unica requisicao e executar fluxos como `review`, `compliance`, `document` e `tests` em sequencia.

Funcionalmente, ele ajuda em cenarios de avaliacao ou automacao nos quais varios artefatos precisam ser processados juntos. A resposta consolida o status do lote e o resultado individual de cada item.

## Review de Pull Request

Endpoint:

- `POST /api/v1/review/pull-request`

Este fluxo amplia o review de codigo para o contexto real de Pull Requests no GitHub. Ele recebe a URL do Pull Request, pode considerar uma chave Jira opcional e analisa o diff com base em criterios como padroes de codigo, seguranca, consistencia com o projeto e aderencia aos criterios da tarefa.

Esse recurso nao era obrigatorio no enunciado original, mas aproxima a solucao do uso cotidiano de um time tecnico.

## Geracao de testes por Pull Request

Endpoint:

- `POST /api/v1/tests/pull-request`

Este fluxo busca o Pull Request no GitHub, infere a linguagem principal pelos arquivos alterados, identifica funcoes, branches e casos de erro criticos no diff e gera testes unitarios focados nas alteracoes.

Ele complementa o fluxo `tests`, que trabalha com codigo enviado diretamente na request.

## Webhook callback

Quando `WEBHOOK_CALLBACK_URL` esta configurado, os fluxos principais podem notificar uma URL externa ao concluir com sucesso ou falha.

O webhook nao substitui a resposta HTTP principal. Ele funciona como mecanismo de integracao para processamentos longos ou automacoes externas, registrando o step do callback no historico.

## Retry configuravel com backoff

As chamadas externas possuem retry configuravel com backoff exponencial. Isso cobre chamadas ao LLM via OpenRouter, consultas de telemetria/precos no OpenRouter, chamadas ao GitHub, consultas ao Jira e callbacks de webhook.

As variaveis `EXTERNAL_RETRY_ATTEMPTS` e `EXTERNAL_RETRY_BASE_DELAY_MS` controlam a quantidade de tentativas e o intervalo base entre elas.

## Rastreabilidade e governanca

---

### Historico de execucoes

Endpoints:

- `GET /api/v1/history`
- `GET /api/v1/history/:id`

O historico registra as execucoes dos fluxos, incluindo tipo, status, payloads, resultado, duracao, cache hit, steps intermediarios e telemetria quando disponivel.

Isso permite auditar como uma resposta foi produzida e facilita depuração, demonstração e avaliação técnica.

## **Analytics de uso**

Endpoint:

- `GET /api/v1/analytics/usage`

O módulo de analytics agrega informações operacionais como uso, tokens e custos. Ele existe para transformar a telemetria persistida em uma visão mais fácil de acompanhar no painel ou em consultas administrativas.

## **Prompts versionados**

Endpoints:

- `GET /api/v1/prompts`
- `GET /api/v1/prompts/:flowType/active`
- `GET /api/v1/prompts/:flowType/:version`
- `POST /api/v1/prompts`
- `POST /api/v1/prompts/:flowType/:version/activate`

Os prompts podem ser cadastrados, consultados e ativados por fluxo. Isso evita que toda evolução de comportamento do agente dependa de alterar código e republicar a aplicação.

Também é possível informar `prompt_version` em requests específicas para usar uma versão diferente da ativa.

## **Catálogo de modelos**

Endpoints:

- `GET /api/v1/models`
- `GET /api/v1/models/default`
- `POST /api/v1/models`
- `PATCH /api/v1/models/:id`

- `DELETE /api/v1/models/:id`

O catalogo de modelos controla quais modelos podem ser usados pela API, qual modelo esta ativo e qual e o padrao global. Requests individuais podem informar `model` para sobrescrever o padrao, desde que o modelo esteja cadastrado e ativo.

## Cache por hash

Os fluxos principais usam hash do payload para reutilizar resultados de execucoes bem-sucedidas quando a entrada e identica. Isso reduz custo, latencia e chamadas repetidas ao provedor LLM.

Mesmo em cache hit, uma nova execucao pode ser registrada apontando para a execucao original, preservando rastreabilidade.

## Experiencia operacional

---

### Healthcheck

Endpoint:

- `GET /health`

O healthcheck permite validar rapidamente se a API subiu. Ele e usado tanto no setup local quanto na validacao via Docker.

### OpenAPI e Swagger UI

Endpoint:

- `GET /docs`

A API publica documentacao interativa para facilitar exploracao dos endpoints, contratos e respostas esperadas.

## Exemplos manuais

O arquivo [requests/cpj-cobranca-api.http](#) concentra exemplos funcionais para uso manual durante desenvolvimento, avaliacao e demonstracao.

## Painel web admin

O painel em [apps/web](#) oferece uma camada visual sobre a API. Ele foi pensado como console tecnico para:

- visualizar dashboard e metricas;
- consultar historico;
- gerenciar prompts;
- gerenciar modelos;
- executar fluxos guiados;
- verificar status da API e acesso ao Swagger.

O painel nao substitui a API. Ele melhora a experiencia de avaliacao e operacao dos recursos implementados.

## Evolucoes fora do escopo obrigatorio nesta versao

---

Os itens abaixo nao eram requisitos obrigatorios do case. Eles aparecem aqui como limites assumidos desta entrega e como caminhos naturais de evolucao:

- autenticao e autorizacao de usuarios finais;
- controle multi-tenant;
- fila assincrona com worker dedicado;
- streaming SSE para todos os fluxos, ja que nesta versao ele foi implementado apenas no fluxo de review;
- edicao visual avancada de prompts;
- avaliacao automatica de qualidade das respostas geradas pelo LLM;
- suporte local via Ollama como provedor padrao.

Esses pontos podem ser evoluídos depois, mas nao prejudicam a avaliacao do case, porque os requisitos obrigatorios foram entregues e varios diferenciais relevantes tambem ja fazem parte da solucao atual.



## Relacao com os proximos capitulos

---

Este capitulo explica o que a solucao faz. Os proximos documentos detalham como ela foi construida:

- [03-stack-e-decisoes.md](#): tecnologias e justificativas.
- [04-arquitetura.md](#): organizacao interna e fluxo tecnico.
- [05-fluxos-de-ia-e-agentes.md](#): engines, graphs, agents e tools.
- [06-api-contratos-e-exemplos.md](#): contratos HTTP e exemplos completos.

# Stack e Decisoes Tecnicas

## Objetivo deste capitulo

Este capitulo explica quais tecnologias foram escolhidas para o case CPJ-Cobrança AI e por que elas fazem sentido para a entrega proposta.

O foco aqui nao e apenas listar ferramentas. A ideia e mostrar como cada decisao ajuda a atender os requisitos do case, reduzir complexidade acidental, melhorar a avaliacao tecnica da entrega e deixar o projeto pronto para evoluir.

## Visao geral da stack

| Camada             | Tecnologia principal        | Papel no projeto                                |
|--------------------|-----------------------------|-------------------------------------------------|
| Runtime backend    | Node.js 22                  | Execucao da API e ferramentas de build/dev      |
| Linguagem backend  | TypeScript 5                | Tipagem, contratos e manutencao                 |
| Framework HTTP     | Fastify 5                   | API REST, plugins, middlewares e OpenAPI        |
| Validacao          | Zod                         | Schemas de entrada e saida                      |
| Persistencia       | Prisma + PostgreSQL 16      | Historico, prompts, modelos, cache e telemetria |
| Orquestracao de IA | LangGraph.js + LangChain.js | Fluxos, agentes, tools e structured output      |
| Provedor LLM       | OpenRouter                  | Acesso a modelos e telemetria de uso/custo      |

|                       |                         |                                        |
|-----------------------|-------------------------|----------------------------------------|
| Observabilidade de IA | LangSmith               | Tracing opcional                       |
| Build backend         | tsup                    | Empacotamento rapido para producao     |
| Testes backend        | Vitest                  | Testes unitarios e de integracao leve  |
| Lint                  | ESLint 9                | Padronizacao e qualidade estatica      |
| Containers            | Docker + Docker Compose | Execucao local e deploy                |
| Frontend              | Next.js 16 + React 19   | Painel administrativo                  |
| UI do frontend        | Ant Design 6            | Componentes visuais do painel          |
| Graficos do painel    | Ant Design Charts       | Custos, tokens e metricas operacionais |

## Leitura da stack sob a otica do case

O case nao exigia uma arquitetura corporativa complexa. Ele exigia algo mais importante: uma solucao que subisse, fosse testavel, tivesse persistencia, integrasse IA de forma util e viesse acompanhada de documentacao clara.

Por isso, as escolhas tecnicas seguiram alguns principios:

- usar tecnologias maduras e amplamente conhecidas;
- reduzir atrito para rodar localmente e em Docker;
- preservar clareza para avaliacao de codigo;
- separar bem responsabilidades sem exagerar na abstracao;
- permitir evolucao real em cima da base entregue.

## Backend

---

### Node.js 22

O backend roda sobre Node.js 22, que oferece um ambiente moderno, estável e adequado para aplicações HTTP, integrações externas e orquestração de fluxos de IA.

Essa escolha ajuda o case em três pontos:

- simplifica o uso de bibliotecas atuais do ecossistema TypeScript;
- conversa bem com Fastify, Prisma, LangChain e LangGraph;
- reduz barreiras para rodar localmente, em CI e em container.

### TypeScript 5

TypeScript foi escolhido para dar previsibilidade ao projeto inteiro: contratos HTTP, estruturas retornadas pelos agentes, modelos persistidos e integrações internas.

No contexto do case, isso importa porque a avaliação considera qualidade de código, clareza arquitetural e confiabilidade da entrega. A tipagem ajuda a:

- documentar intenções no próprio código;
- reduzir ambiguidade entre schema, implementação e testes;
- facilitar manutenção conforme novos fluxos forem adicionados.

O projeto combina TypeScript com Zod para aproximar tipo estático e validação em runtime, evitando depender apenas do compilador para entradas externas.

### Fastify 5

Fastify foi escolhido como framework da API por ser enxuto, rápido e muito bom para projetos modulares baseados em plugins, rotas e schemas.

Ele atende bem ao case porque permite:

- montar uma API REST clara e organizada;
- registrar middlewares, CORS, rate limit e segurança com pouco atrito;

- integrar Swagger/OpenAPI de forma nativa;
- manter o código de bootstrap pequeno e legível.

Em uma entrega de case, essa clareza pesa bastante: o avaliador consegue entender facilmente como a aplicação sobe, como as rotas entram e como os módulos se conectam.

## Zod

Zod foi usado para validar payloads de entrada e estruturar dados de saída dos fluxos. Essa escolha conversa diretamente com um dos riscos naturais de projetos com LLM: receber dados livres demais ou respostas inconsistentes.

Com Zod, a aplicação ganha:

- validação previsível nas bordas da API;
- contratos reutilizáveis entre backend, testes e documentação;
- menor risco de aceitar requests inválidas ou propagar estruturas quebradas.

## Persistência

---

### PostgreSQL 16

PostgreSQL foi escolhido como banco relacional principal porque o projeto precisa guardar histórico de execuções, steps, telemetria, prompts versionados, catálogo de modelos e dados de batch de forma consistente.

Para o case, um banco relacional faz sentido porque:

- o histórico precisa ser consultável e auditável;
- prompts e modelos possuem relações e estados bem definidos;
- a avaliação valoriza modelagem coerente e persistência real.

## Prisma

Prisma foi adotado como camada de acesso a dados. A escolha equilibra produtividade, legibilidade e segurança de tipos sem esconder totalmente a modelagem do banco.

Ele foi especialmente útil aqui para:

- manter schema e código alinhados;
- simplificar migrations e seed inicial;
- facilitar queries de histórico, analytics e governança;
- reduzir a quantidade de SQL manual no fluxo principal.

O uso de Prisma também melhora a experiência do avaliador, porque o schema fica centralizado e mais fácil de ler do que uma camada de persistência espalhada em strings SQL soltas.

## IA e orquestração

---

### LangGraph.js

LangGraph foi escolhido para modelar os fluxos de IA como grafos explícitos, e não como uma única chamada monolítica ao modelo.

Isso combina com o problema do case porque vários fluxos possuem etapas distintas: validações, roteamento por linguagem, tools determinísticas, especialistas e agregação final.

Os ganhos principais são:

- visibilidade de etapas;
- melhor separação entre responsabilidades;
- possibilidade de registrar steps no histórico;
- facilidade para evoluir fluxos sem reescrever tudo.

### LangChain.js

LangChain foi usado como camada de integração com modelo, structured output e composição de comportamento dos agentes.

Neste projeto, ele funciona como complemento do LangGraph:

- LangGraph organiza o fluxo;
- LangChain ajuda a padronizar a conversa com o LLM.

Essa combinacao foi uma escolha boa para o case porque permitiu construir algo mais robusto do que um wrapper simples de `fetch`, sem transformar a entrega em uma plataforma excessivamente abstrata.

## OpenRouter

OpenRouter foi escolhido como provedor LLM por permitir acesso a diferentes modelos por uma interface unica, mantendo flexibilidade de custo e troca de modelo.

Para o case, isso tem varias vantagens:

- facilita documentar modelo padrao e alternativas;
- reduz acoplamento a um unico vendor;
- combina bem com o catalogo de modelos persistido no banco;
- permite capturar metadados de uso e custo quando disponiveis.

O projeto ainda adiciona retry configuravel com backoff nas chamadas externas, o que melhora resiliencia sem exigir um provedor proprietario adicional.

## LangSmith

LangSmith entra como tracing opcional. Ele nao e necessario para rodar o projeto, mas foi mantido como diferencial para observabilidade de fluxos de IA.

Essa decisao e importante porque equilibra dois objetivos:

- oferecer um caminho de diagnostico mais avancado;
- nao bloquear setup local ou avaliacao quando a chave nao estiver disponivel.

## Qualidade de codigo e entrega

---

### Vitest

Vitest foi escolhido para testes por ter boa ergonomia no ecossistema TypeScript, execucao rapida e sintaxe simples.

Ele cobre bem o que o case precisa demonstrar:

- validacao de contratos;
- verificacao de rotas;
- testes de servicos e repositórios;
- protecao contra regressao em integrações e config.

## ESLint 9

ESLint ajuda a manter consistencia e legibilidade do codigo. Em um case tecnico, isso importa nao apenas por padrao de estilo, mas porque pequenas inconsistencias acabam dificultando leitura e revisao.

## tsup

**tsup** foi escolhido para o build do backend por ser simples, rapido e suficiente para uma API Node.js deste porte.

Essa decisao evita um pipeline de build pesado demais e deixa claro que a prioridade aqui e a entrega funcional do servico, nao uma sofisticacao desnecessaria no empacotamento.

## Frontend e experiencia operacional

---

### Next.js 16 + React 19

O painel web foi construido com Next.js e React para oferecer uma interface tecnica de apoio ao avaliador e ao operador da API.

Essa escolha foi util porque:

- facilita criar paginas administrativas rapidamente;
- funciona bem em uma estrutura de workspace junto com a API;
- entrega uma interface moderna sem exigir uma aplicacao frontend isolada e excessivamente customizada.

No contexto do case, o painel nao substitui a API, mas melhora muito a avaliacao da entrega ao mostrar historico, prompts, modelos, status e execucao guiada dos fluxos.



## Ant Design 6

Ant Design foi escolhido para acelerar a construção de uma interface com cara de ferramenta operacional, e não de landing page.

Isso combina com o público do projeto:

- avaliador técnico;
- desenvolvedor;
- pessoa operando os fluxos ou inspecionando histórico.

A biblioteca oferece tabelas, formulários, layout, componentes de feedback e elementos administrativos prontos, o que ajudou a manter foco na funcionalidade em vez de gastar tempo excessivo com design do zero.

## Ant Design Charts

Os gráficos foram usados para apoiar a visualização de custos, tokens e uso no painel. É uma escolha pragmática: reaproveita o ecossistema visual da própria stack do frontend e acelera a entrega de métricas operacionais.

# Containers, setup e deploy

---

## Docker e Docker Compose

Docker e Docker Compose foram escolhidos porque o case exigia uma entrega containerizada e com setup reproduzível.

Essa escolha ajuda diretamente nos critérios de avaliação:

- sobe API, banco e painel de forma consistente;
- reduz diferenças entre ambiente local e ambiente de deploy;
- simplifica demonstração;
- facilita automação no GitHub Actions e deploy em VPS.

Também foi uma decisão importante para confiabilidade da entrega: quando a documentação diz `docker compose up --build`, o projeto de fato foi preparado para seguir esse caminho.

## Trade-offs assumidos

Nem toda escolha foi sobre maximizar poder tecnico. Algumas foram sobre manter a solucao avaliavel e sustentavel dentro do tempo do case.

Os principais trade-offs foram:

- **usar Fastify em vez de um framework mais opinativo:** menos estrutura pronta, mas mais controle e menos peso;
- **usar Prisma em vez de SQL manual em toda a aplicacao:** mais produtividade e legibilidade, com menor controle fino em consultas muito especificas;
- **usar OpenRouter em vez de integrar varios providers diretamente:** mais simplicidade operacional, com dependencia de um gateway unico;
- **usar painel admin com Ant Design em vez de frontend altamente customizado:** mais velocidade de entrega e melhor foco no uso tecnico;
- **usar LangGraph/LangChain sem exagerar em camadas proprietarias:** melhor estrutura para os fluxos, preservando legibilidade do codigo.

## Decisao central da stack

Se fosse resumir a logica da stack em uma frase, seria esta:

construir uma base tecnica moderna, testavel e rastreavel, usando ferramentas conhecidas o suficiente para facilitar avaliacao e evolucao, sem transformar o case em uma arquitetura maior do que o problema pedia.

## Relacao com os proximos capitulos

Depois deste capitulo, a leitura natural e:

- [04-arquitetura.md](#), para entender como essa stack foi organizada no codigo;
- [05-fluxos-de-ia-e-agentes.md](#), para detalhar o papel de LangGraph, LangChain, tools, especialistas e aggregator;
- [10-como-rodar.md](#), para ver a stack em execucao local e em Docker.

# Arquitetura

---

## Objetivo deste capítulo

---

Este capítulo descreve como o projeto foi organizado internamente para atender o case técnico do CPJ-Cobrança AI. O foco aqui é mostrar a estrutura do sistema como arquitetura de software: camadas, módulos, fluxo de uma request, responsabilidades e pontos de integração.

O objetivo não é detalhar cada algoritmo ou cada prompt. A ideia é permitir que um avaliador técnico entenda rapidamente:

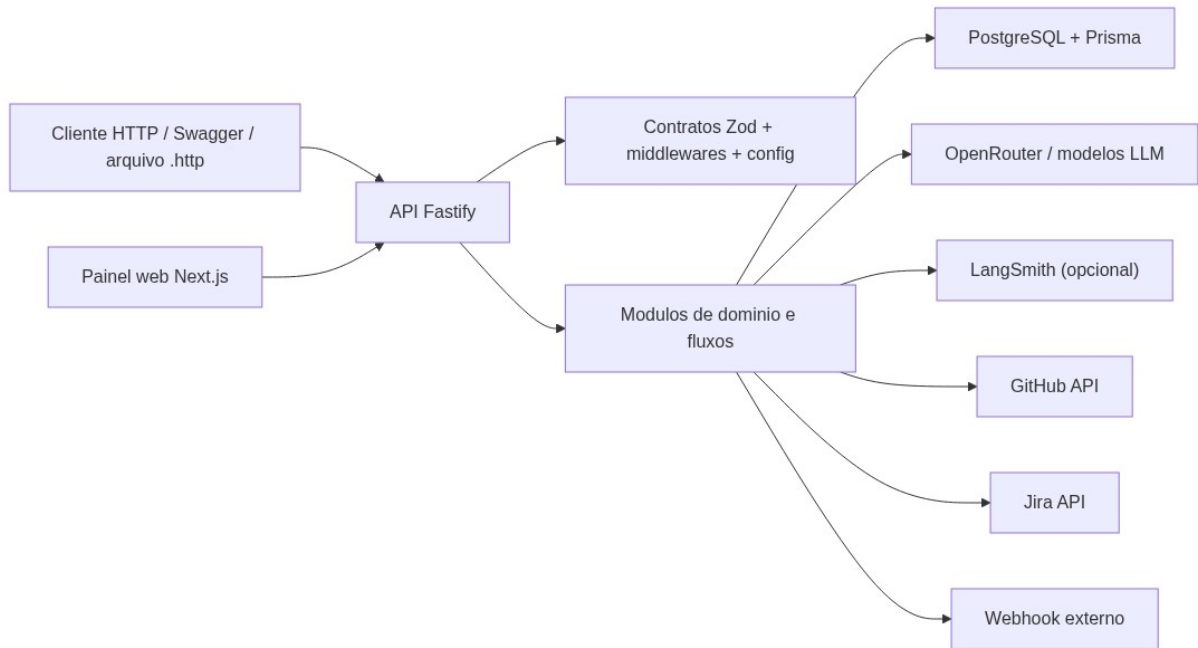
- como a aplicação sobe;
- como as dependências são montadas;
- como os fluxos foram separados;
- como a API conversa com banco, LLM e painel web;
- onde estão os principais pontos de extensão e rastreabilidade.

## Visão arquitetural

---

Em alto nível, a solução é composta por quatro blocos principais:

- **cliente:** consumidores HTTP e painel web;
- **API backend:** camada principal de orquestração e execução dos fluxos;
- **persistência:** PostgreSQL via Prisma para histórico, telemetria, prompts, modelos e batch;
- **integrações externas:** OpenRouter, LangSmith opcional, GitHub, Jira e webhook callback.



## Principio estrutural da arquitetura

A arquitetura foi desenhada para equilibrar tres necessidades do case:

- **clareza para avaliacao tecnica;**
- **separacao de responsabilidades;**
- **entrega real, rodavel e extensivel.**

Por isso, o projeto nao foi modelado como um unico servico chamando LLM de forma direta. Em vez disso, cada capacidade foi organizada em modulos com fronteiras claras, contratos compartilhados e uma camada de engine responsavel por executar o fluxo real.

## Camadas da aplicacao

### 1. Entrada e composicao

Os arquivos `src/server.ts` e `src/app.ts` sao o ponto de entrada da aplicacao.

Nessa camada acontecem quatro coisas principais:

- carregamento de variaveis de ambiente;
- criacao da instancia Fastify;

- registro de plugins e middlewares globais;
- registro das rotas por modulo.

Essa decisao centraliza o bootstrap e evita espalhar configuracao critica pelo projeto.

## 2. Shared

O directorio `src/shared/` concentra tudo o que atravessa varios modulos:

- `config/`: leitura e validacao de ambiente;
- `contracts/`: schemas Zod e tipos compartilhados;
- `middlewares/`: tratamento global de erro, contexto de request e seguranca;
- `utils/`: hash, datas e retry compartilhado.

Essa camada existe para evitar duplicacao e manter consistencia entre rotas, servicos, testes e documentacao.

## 3. Infrastructure

O directorio `src/infrastructure/` concentra integracoes transversais que nao pertencem a um fluxo especifico:

- `database/`: plugin Prisma para o Fastify;
- `logging/`: configuracao de logger;
- `openapi/`: registro de Swagger/OpenAPI;
- `errors/`: tratamento e normalizacao de erros.

E uma camada de apoio tecnico. Ela sustenta a API, mas nao concentra regra de negocio nem detalhes funcionais dos fluxos.

## 4. Modules

O directorio `src/modules/` concentra os blocos funcionais reais do sistema.

Ele foi dividido em dois grandes grupos:

- **modulos de fluxo:** `review`, `compliance`, `document`, `tests`, `batch`;

- **modulos de apoio e governanca:** `history`, `analytics`, `prompts`, `models`, `executions`, `agent`.

Essa divisao ajuda a separar:

- o que entrega valor funcional direto ao case;
- o que viabiliza rastreabilidade, configuracao e observabilidade.

## Organizacao por modulo

---

Cada modulo segue, em maior ou menor grau, o mesmo padrao estrutural:

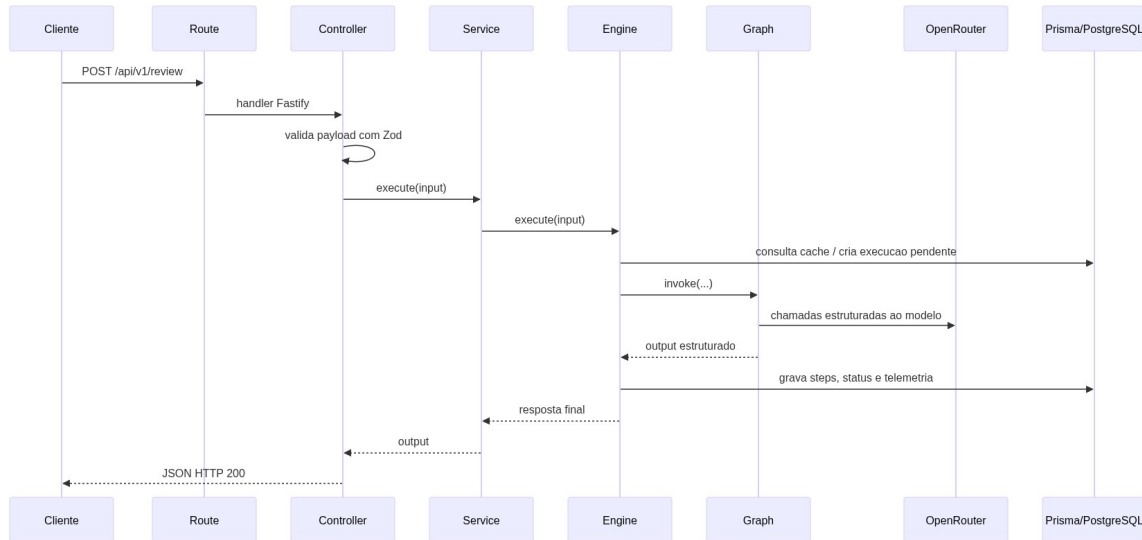
- `routes/`: registra endpoints Fastify;
- `controllers/`: recebe request, valida payload e envia response;
- `services/`: coordena o uso do fluxo a partir da perspectiva HTTP;
- `engines/`: executa a logica principal do fluxo;
- `graphs/`: define orquestracao em LangGraph quando aplicavel;
- `agents/`: encapsula especialistas baseados em LLM;
- `tools/`: validacoes ou analises deterministicas;
- `prompts/`: catalogos e templates de prompt;
- `docs/`: schemas OpenAPI por rota.

Nem todo modulo possui todas essas pastas, mas o padrao geral se repete.

## Fluxo de uma request

---

O ciclo de request segue uma sequencia clara da borda HTTP ate a persistencia e o retorno estruturado.



Esse fluxo também ajuda a entender uma decisão importante da arquitetura: o controller não conhece detalhes de LLM, banco ou grafo. Ele apenas recebe, valida e delega.

## Papel de cada camada no request lifecycle

### Routes

As rotas fazem a ligação entre a URL HTTP e o controller correspondente. Também registram schemas OpenAPI e configurações do Fastify, como `attachValidation`.

Exemplo concreto: `src/modules/review/routes/index.ts`

### Controllers

Os controllers transformam a request HTTP em chamada de aplicação. Eles:

- parseiam e validam payloads;
- convertem erro de validação em resposta apropriada;
- chamam o service correto;
- entregam a resposta HTTP final.

Exemplo concreto: `src/modules/review/controllers/index.ts`

### Services

Os services funcionam como fachada do fluxo para a camada HTTP. Eles escolhem qual engine usar, resolvem modo de execucao especial quando necessario e expõem uma interface mais simples para o controller.

No caso de `review`, por exemplo, o service tambem trata o modo streaming SSE sem jogar essa responsabilidade para o engine inteiro.

## Engines

Os engines sao o centro operacional dos fluxos. Eles concentram:

- verificacao de cache;
- criacao de execucao pendente;
- chamada do grafo principal;
- marcacao de sucesso ou falha;
- gravacao de steps e telemetria;
- notificacao de webhook quando configurado.

Exemplo concreto: `src/modules/review/engines/review.engine.ts`

## Graphs

Os graphs modelam a orquestracao do fluxo. Eles existem porque varios fluxos de IA nao sao apenas uma chamada unica ao modelo: ha selecao de caminho, ferramentas deterministicas, agentes especialistas e agregacao.

Essa camada e especialmente importante no `review`, que possui:

- roteamento por linguagem;
- grafos especificos por linguagem;
- especialistas por criterio de avaliacao;
- aggregator final.

## Agents e tools

Os agents encapsulam especialistas baseados em LLM. As tools executam analises deterministicas que complementam ou guiam o fluxo.



Essa combinacao foi escolhida para evitar depender 100% de comportamento livre do modelo. A arquitetura mistura heurísticas previsíveis com análise generativa.

## Dependencias e composicao

---

A composicao principal acontece em `App`, mas cada modulo tambem sabe criar suas proprias dependencias padrao quando recebe um `FastifyInstance` com Prisma registrado.

Isso gera um modelo leve de injecao de dependencias:

- no runtime real, as rotas constroem services e repositories concretos;
- em testes, as dependencias podem ser injetadas manualmente;
- a aplicacao continua simples, sem precisar de um container DI pesado.

Esse padrao aparece com clareza em `src/modules/review/routes/index.ts`, onde o modulo cria `DefaultReviewService`, `ReviewExecutionRepository`, `DefaultPromptsService` e `DefaultModelsService` quando necessario.

## Persistencia na arquitetura

---

A persistencia foi desenhada como parte estrutural da solucao, nao como detalhe acoplado ao fim do fluxo.

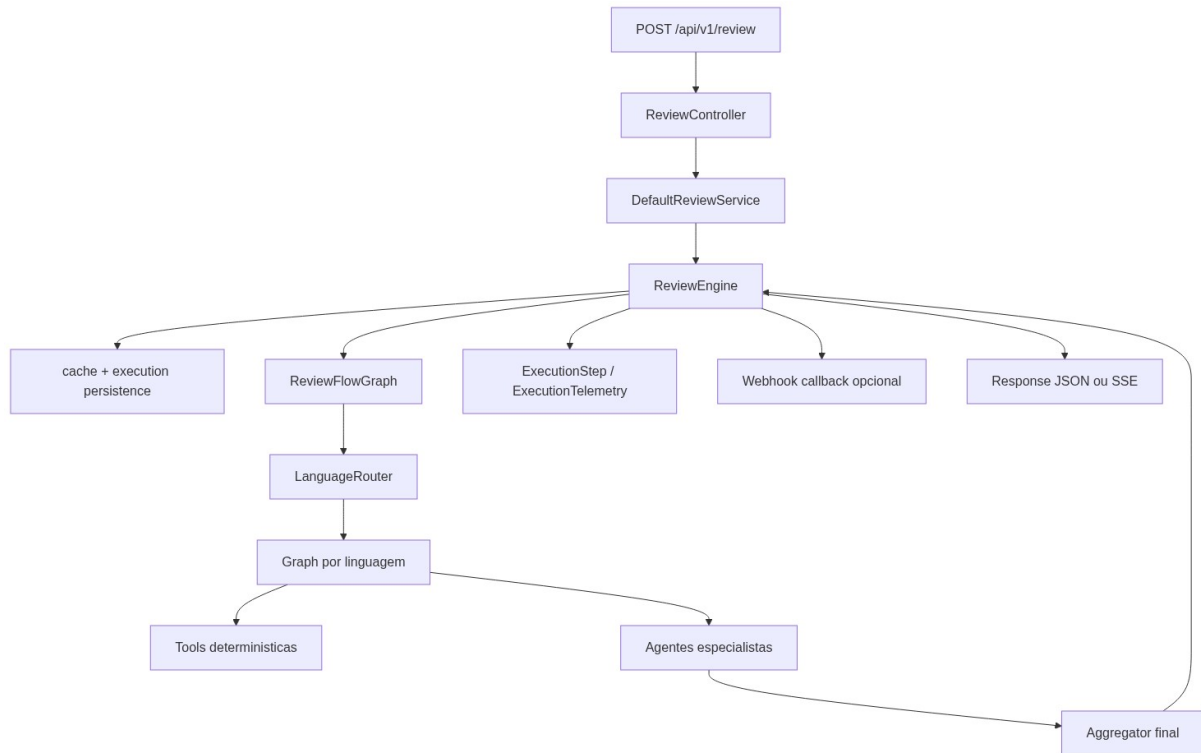
O schema Prisma guarda:

- `Execution`: execucao principal do fluxo;
- `ExecutionStep`: steps intermediarios;
- `ExecutionTelemetry`: uso de modelo, tokens e custo;
- `PromptVersion`: versoes de prompt por fluxo e bloco;
- `RegisteredModel` e `GlobalModelSettings`: governanca de modelos;
- `BatchExecution`: resumo de execucoes em lote.

Essa modelagem deixa a arquitetura mais auditavel e explica por que a aplicacao nao trata cada resposta do LLM como algo descartavel.

## Arquitetura do review como exemplo de referencia

O fluxo **review** é o melhor exemplo para entender a arquitetura inteira, porque ele exercita quase todos os componentes do sistema ao mesmo tempo.



Essa estrutura se repete, com variações menores, em **compliance**, **document** e **tests**.

## Arquitetura do painel web

O frontend em **apps/web** não replica a lógica dos fluxos. Ele funciona como cliente da API.

Em termos arquiteturais, isso é importante porque preserva uma separação clara:

- a API é a fonte de verdade funcional;
- o painel é uma camada de operação e visualização.

Isso reduz duplicação de regra e evita que parte da lógica do caso fique espalhada entre backend e frontend.

## Middlewares e preocupações transversais

Algumas responsabilidades aparecem em toda a aplicação e por isso foram centralizadas em middlewares e infraestrutura compartilhada:

- tratamento global de erros;
- contexto de request;
- segurança HTTP;
- OpenAPI/Swagger;
- logger;
- conexão Prisma.

Esse desenho evita repetir setup técnico em cada módulo e ajuda a manter a arquitetura coesa.

## Por que essa arquitetura funciona bem para o case

---

Esta arquitetura foi uma boa escolha para o case por cinco motivos:

1. deixa o bootstrap simples e fácil de auditar;
2. separa HTTP, execução de fluxo, persistência e integrações externas;
3. permite testes por camada;
4. acomoda bem fluxos com IA sem reduzir tudo a uma única chamada de prompt;
5. cria base real para evolução futura sem inflar demais a entrega.

## Limites assumidos na arquitetura atual

---

A arquitetura foi pensada para uma entrega de case e, por isso, assume alguns limites conscientes:

- não há mensageria externa nem fila dedicada;
- os fluxos rodam de forma síncrona no backend principal;
- não existe separação por microserviços;
- o painel web depende integralmente da API para dados operacionais;
- o modelo de injeção de dependências é manual, e não baseado em framework DI.

Esses limites não empobrecem a entrega. Eles mantêm a solução legível, demonstrável e suficiente para o escopo pedido.

## Relação com os próximos capítulos

---

Depois deste capítulo, os próximos documentos aprofundam dois pontos específicos desta arquitetura:

- [05-fluxos-de-ia-e-agentes.md](#): detalha graphs, tools, especialistas e aggregator;
- [07-dados-cache-telemetria.md](#): detalha a modelagem persistida no banco.

# Fluxos de IA e Agentes

---

## Objetivo deste capítulo

---

Este capítulo detalha como os fluxos de IA funcionam dentro do projeto. O foco não é apenas dizer que a API "chama um modelo", mas explicar como a aplicação combina:

- tools determinísticas;
- prompts versionados;
- agentes especialistas;
- structured output;
- grafos LangGraph;
- persistência de steps;
- telemetria e retry.

Esse é o coração técnico da entrega, porque é aqui que a solução deixa de ser um wrapper simples de LLM e passa a operar como um conjunto de fluxos especializados.

## Visão geral dos fluxos

---

Os fluxos com IA do projeto podem ser lidos em três grupos:

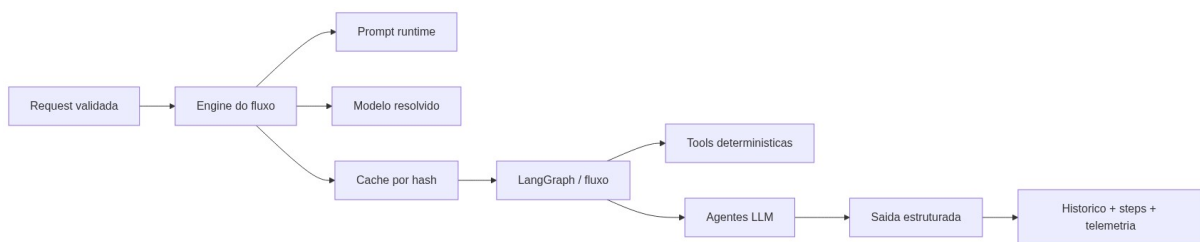
- **fluxo de review**, que é o mais rico e usa roteamento por linguagem, especialistas e aggregator;
- **fluxos lineares**, como **compliance**, **document** e **tests**, que seguem a ideia tool -> agente;
- **fluxos de Pull Request**, que combinam integrações externas, preparação de payload e avaliação estruturada.

## Padrão geral de execução

---

Apesar das diferenças entre os fluxos, existe um padrão arquitetural comum:

1. a request entra pelo controller e chega ao service;
2. o engine resolve prompt e modelo;
3. o engine consulta cache quando aplicavel;
4. o grafo executa steps determinísticos e/ou agentes LLM;
5. as respostas retornam em formato estruturado;
6. o engine persiste status, steps, telemetria e callback de webhook;
7. a API devolve o resultado final.



## Structured output como base do comportamento

Um dos pilares da solução é o uso de saída estruturada em vez de respostas textuais livres. A classe `src/modules/agent/llm/structured-output.runner.ts` faz esse papel.

Na prática, isso significa que cada agente ou aggregator:

- recebe mensagens `system` e `user`;
- responde segundo um schema Zod esperado;
- passa por validação antes de a resposta seguir no fluxo.

Essa escolha reduz ambiguidades, melhora a confiabilidade da API e faz com que os contratos dos endpoints conversem melhor com testes, OpenAPI e persistência.

## Tools determinísticas versus agentes LLM

O projeto não delega tudo ao modelo. Ele combina duas categorias de execução:

### Tools determinísticas

As tools são responsáveis por sinais objetivos e previsíveis. Elas rodam sem LLM e normalmente operam com heurísticas, regexes ou análise simples do input.

Exemplo no review:

- detecção de logs com dados sensíveis;
- detecção de interpolação SQL evidente.

Exemplo em outros fluxos:

- extração de requisitos em **compliance**;
- sinais de API pública em **document**;
- candidatos de comportamento em **tests**.

## Agentes LLM

Os agentes entram quando a tarefa exige interpretação, consolidação de contexto ou julgamento técnico mais semântico.

Eles recebem:

- o input original;
- o contexto da linguagem;
- os achados das tools;
- o prompt do bloco correspondente.

Essa divisão é importante porque evita tratar o LLM como oráculo absoluto.

## Fluxo de review

---

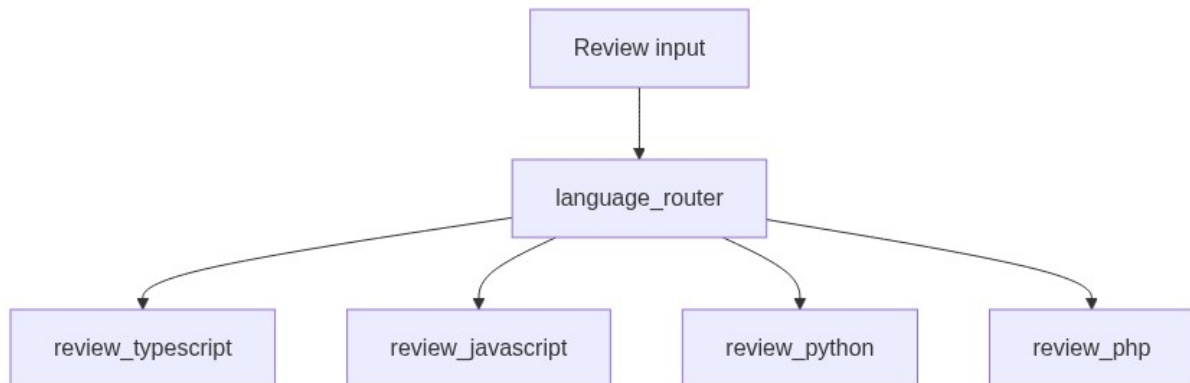
O fluxo **review** é o mais completo do sistema e funciona como referência para o resto da arquitetura de IA.

### Etapa 1: roteamento por linguagem

O **ReviewFlowGraph** recebe a request e primeiro passa pelo node **language\_router**.

Essa etapa identifica a linguagem do input e direciona para um dos grafos específicos:

- `review_typescript`
- `review_javascript`
- `review_python`
- `review_php`



Isso permite adequar perfis, contexto e leitura do código sem misturar regras de linguagens diferentes em um único fluxo genérico.

## Etapa 2: tools determinísticas

Dentro do grafo da linguagem, o primeiro passo é `deterministic_tools`.

Esse passo gera achados objetivos que depois alimentam os agentes especialistas. Em vez de pedir ao LLM que redescubra tudo do zero, a aplicação já entrega parte do terreno preparado.

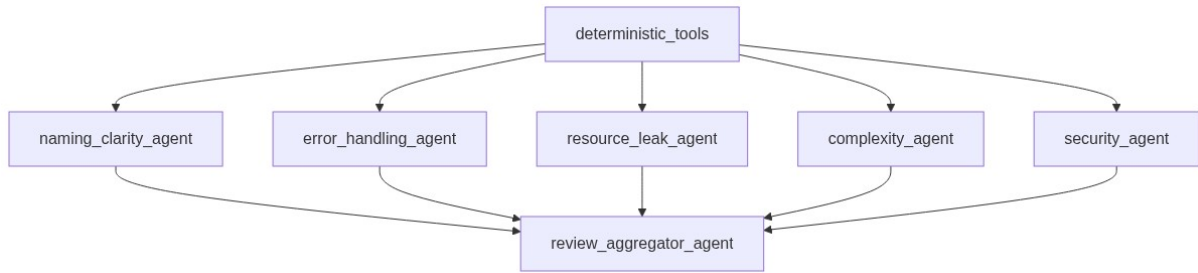
## Etapa 3: especialistas paralelos

Depois das tools, o fluxo dispara especialistas independentes:

- `naming_clarity_agent`
- `error_handling_agent`
- `resource_leak_agent`
- `complexity_agent`
- `security_agent`

Cada um recebe o mesmo contexto base, mas com uma missão diferente.





Essa abordagem traz duas vantagens:

- reduz a chance de uma resposta unica ignorar dimensoes importantes;
- deixa a analise mais observavel e modular.

## Etapa 4: aggregator final

O `review_aggregator_agent` recebe:

- código original;
- contexto opcional do usuário;
- achados determinísticos;
- saídas dos especialistas.

Ele é responsável por produzir a resposta final do endpoint, no schema `ReviewResponse`.

Esse aggregator não repete simplesmente os especialistas. Ele consolida, remove redundância, organiza severidades e constrói um parecer único.

## Prompts no review

O review usa múltiplos blocos de prompt:

- um por especialista;
- um para o aggregator.

Esses blocos podem vir de duas fontes:

- conjunto versionado armazenado no banco;

- fallback legado em catalogos locais quando nao ha resolver persistido.

Isso permite ajustar comportamento dos especialistas sem precisar alterar toda a estrutura do fluxo.

## Fluxos lineares: compliance, document e tests

Os fluxos `compliance`, `document` e `tests` seguem uma estrategia mais simples do que o `review`. Em todos eles, o padrao geral e:

- uma tool prepara sinais ou estrutura parcial;
- um agente LLM gera a resposta final.

### Compliance

O `ComplianceFlowGraph` roda:

1. `requirements_extractor`
2. `compliance_agent`



O primeiro passo extrai requisitos e sinais relevantes; o segundo cruza isso com o código e produz score, requisitos cobertos, faltantes e parciais.

### Document

O `DocumentFlowGraph` roda:

1. `document_signal_extractor`
2. `document_agent`

Aqui a tool tenta identificar candidatos de API publica, sinais de entrada, saída e comportamento, e o agente monta a documentacao final.

## Tests

O `TestsFlowGraph` roda:

1. `tests_signal_extractor`
2. `tests_agent`

Nesse caso, a tool destaca comportamentos, caminhos e sinais que merecem teste, e o agente devolve o arquivo de testes, casos cobertos e dicas de cobertura.

## Fluxo de review de Pull Request

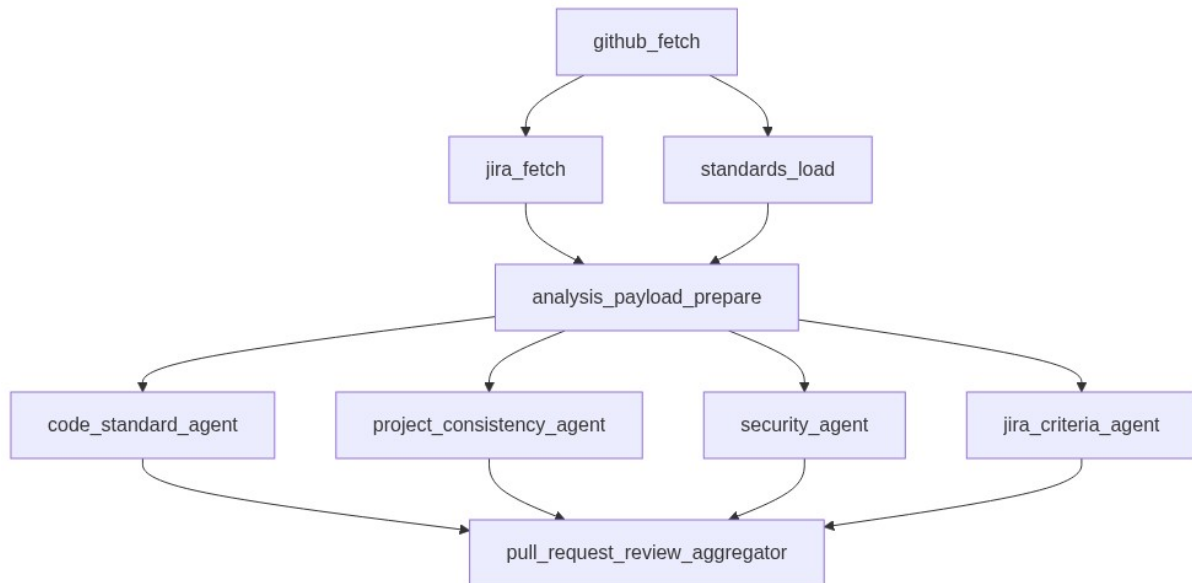
---

O fluxo `review/pull-request` amplia o modelo tradicional de review para um cenário de repositório real.

Ele combina:

- busca de dados no GitHub;
- busca opcional no Jira;
- carga de padrões locais do projeto;
- preparação de payload truncado;
- análise por seções;
- decisão agregada final.

### Etapas do fluxo



### O que esse fluxo ganha com isso

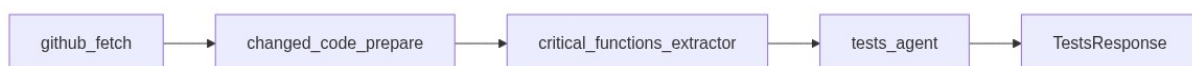
- o review deixa de olhar apenas um trecho solto de código;
- o diff é analisado com contexto de projeto;
- há cruzamento com critérios de Jira quando informados;
- o sistema retorna seções distintas e um veredito consolidado.

O node `jira_criteria_agent` pode ser marcado como `skipped` quando não existe `jira_issue_key`, o que mantém o fluxo determinístico sem forçar uma integração obrigatória.

## Fluxo de testes por Pull Request

O fluxo `tests/pull-request` também usa GitHub como fonte, mas com objetivo diferente: gerar testes focados nas alterações mais críticas do diff.

### Etapas do fluxo



### Comportamentos relevantes

Esse fluxo:

- infere a linguagem principal pelos arquivos alterados;
- trunca payloads grandes para caber no contexto do modelo;
- extrai funcoes, classes, branches e casos de erro criticos;
- usa esse material para orientar a geracao do arquivo de teste final.

Ou seja: nao e apenas "gerar teste para um PR". Existe uma etapa intermediaria de preparacao e priorizacao do que realmente merece cobertura.

## Retry com backoff nas chamadas de IA e integracoes

---

As chamadas externas nao dependem de tentativa unica. O projeto aplica retry configuravel com backoff em pontos como:

- execucoes LLM via `LangChainStructuredOutputRunner`;
- telemetria/precos do OpenRouter;
- GitHub;
- Jira;
- webhook callback.

Isso melhora resiliencia para falhas transitorias sem poluir cada fluxo com logica repetida de repeticao manual.

## Telemetria dos fluxos

---

Os fluxos com LLM usam `AgentTelemetryCollector` para acumular dados de uso de modelo ao longo da execucao.

Ao final, o engine pode persistir:

- provider;
- modelo solicitado;
- modelo realmente usado;
- generation ids;
- tokens de entrada e saida;

- custo total, entrada e saída;
- cache read tokens.

Essa camada é importante porque transforma o uso do LLM em algo mensurável, e não apenas observável por texto de log.

## Steps persistidos como trilha de execução

---

Cada grafo registra steps intermediários quando existe `executionId` e `stepRecorder` disponíveis.

Isso significa que a aplicação guarda não apenas o resultado final, mas também etapas como:

- `language_router`
- `deterministic_tools`
- `security_agent`
- `review_aggregator_agent`
- `github_fetch`
- `analysis_payload_prepare`
- `tests_signal_extractor`

Esse desenho melhora muito a auditabilidade do case e ajuda a explicar a resposta final com mais precisão.

## Webhook e camada de engine

---

Os grafos cuidam da execução interna do fluxo. Já o engine fica responsável por atividades transversais ao redor dele:

- cache;
- persistência de execução;
- mark success / failed;
- telemetria agregada;
- webhook callback.

Isso é importante arquiteturalmente porque evita misturar detalhes de entrega externa dentro dos nodes do grafo.

## Diferença entre grafo e engine

---

Vale registrar de forma bem objetiva:

- **grafo**: descreve como o fluxo pensa;
- **engine**: descreve como o fluxo opera dentro da aplicação.

O grafo decide etapas, especialistas, tools e agregação. O engine decide cache, persistência, telemetria, webhook e integração com o ciclo de execução da API.

## Por que essa abordagem faz sentido no case

---

Essa modelagem foi uma boa escolha para o case porque:

1. evita endpoints opacos demais;
2. permite explicar o comportamento dos agentes com clareza;
3. melhora testabilidade por step e por fluxo;
4. cria base para evoluir prompts, modelos e nodes sem reescrever tudo;
5. entrega algo mais próximo de um sistema de IA operacional do que de um protótipo isolado.

## Relação com os próximos capítulos

---

Depois deste capítulo, a leitura natural é:

- [06-api-contratos-e-exemplos.md](#), para ver como esses fluxos aparecem nos endpoints;
- [07-dados-cache-telemetria.md](#), para entender como cache, steps e telemetria ficam persistidos;

- `08-prompts-modelos-governanca.md`, para detalhar como prompts e modelos são resolvidos em runtime.



# API, Contratos e Exemplos

---

## Objetivo deste capítulo

---

Este capítulo documenta a superfície HTTP da aplicação: endpoints, payloads, respostas, filtros, SSE e exemplos práticos.

O foco é ajudar quem avalia o caso a entender:

- quais rotas existem;
- o que cada uma recebe;
- o que cada uma retorna;
- onde entram `model` e `prompt_version`;
- como os endpoints se relacionam com histórico, batch e governança.

## Convencões gerais da API

---

### Base path

As rotas de negócio ficam sob `/api/v1`, com exceção de:

- `GET /health`
- `GET /docs`

### Formato de payload

As requests e responses usam JSON, exceto o endpoint de streaming do review, que responde como `text/event-stream`.

### Validação

As entradas são validadas com Zod. Quando o payload não atende ao schema esperado, a API responde com erro de validação em vez de deixar o fluxo seguir com dados ambíguos.

## Overrides por request

Nos fluxos com IA, dois campos podem aparecer como opcionais:

- `prompt_version`: força uma versão específica de prompt para aquela execução;
- `model`: sobrescreve o modelo padrão global, desde que ele esteja cadastrado e ativo.

## Mapa geral dos endpoints

| Metodo | Rota                                     | Finalidade                        |
|--------|------------------------------------------|-----------------------------------|
| GET    | <code>/health</code>                     | Healthcheck da API                |
| POST   | <code>/api/v1/review</code>              | Review de código                  |
| POST   | <code>/api/v1/review/stream</code>       | Review via SSE                    |
| POST   | <code>/api/v1/review/pull-request</code> | Review de Pull Request            |
| POST   | <code>/api/v1/compliance</code>          | Aderência entre tarefa e código   |
| POST   | <code>/api/v1/document</code>            | Geração de documentação           |
| POST   | <code>/api/v1/tests</code>               | Geração de testes                 |
| POST   | <code>/api/v1/tests/pull-request</code>  | Geração de testes a partir de PR  |
| POST   | <code>/api/v1/batch</code>               | Execução de vários fluxos em lote |
| GET    | <code>/api/v1/history</code>             | Lista de execuções                |
| GET    | <code>/api/v1/history/:id</code>         | Detalhe de uma execução           |

|        |                                             |                                  |
|--------|---------------------------------------------|----------------------------------|
| GET    | /api/v1/analytics/usage                     | Agregados de uso, tokens e custo |
| GET    | /api/v1/prompts                             | Lista versoes de prompt          |
| GET    | /api/v1/prompts/:flowType/active            | Busca prompt ativo do fluxo      |
| GET    | /api/v1/prompts/:flowType/:version          | Busca versao especifica          |
| POST   | /api/v1/prompts                             | Cria nova versao de prompt       |
| POST   | /api/v1/prompts/:flowType/:version/activate | Ativa uma versao                 |
| GET    | /api/v1/models                              | Lista modelos cadastrados        |
| GET    | /api/v1/models/default                      | Busca modelo padrao global       |
| POST   | /api/v1/models                              | Cadastra modelo                  |
| PATCH  | /api/v1/models/:id                          | Atualiza modelo                  |
| DELETE | /api/v1/models/:id                          | Remove modelo nao padrao         |
| GET    | /docs                                       | Swagger UI                       |

## Endpoints de fluxo

### POST /api/v1/review

Faz review estruturado de um trecho de código.

#### Request

```
{
  "code": "function sum(a, b) { return a + b; }"
```

```
{
  "language": "typescript",
  "context": "Trecho usado em um modulo financeiro",
  "prompt_version": 1,
  "model": "openai/gpt-4o-mini"
}
```

## Response

```
{
  "overall_quality": "needs_improvement",
  "score": 7,
  "issues": [
    {
      "severity": "medium",
      "line_hint": "linha 1",
      "description": "A funcao nao explicita contrato de tipos de entrada.",
      "suggestion": "Defina tipos ou valide entradas conforme o contrato esperado."
    }
  ],
  "positives": [
    "Funcao pequena e facil de entender."
  ],
  "summary": "O codigo e simples, mas merece um contrato mais claro."
}
```

## Campos principais

- `language`: `typescript`, `javascript`, `python` ou `php`
- `context`: opcional
- `prompt_version`: opcional
- `model`: opcional

## POST `/api/v1/review/stream`

Executa o mesmo review, mas em SSE.

## Request

Usa o mesmo payload do endpoint `/api/v1/review`.

## Eventos SSE

Os eventos emitidos são:

- `started`
- `step`
- `result`
- `error`
- `done`

### Exemplo de sequência

```
event: started
data: {"execution_id":"uuid","cache_hit":false}
event: step
data: {"node_name":"language_router","kind":"system","status":"success","duration_ms":8}
event: result
data: {"output":{"overall_quality":"good","score":9,"issues":[],"positives":["Codigo simpl
event: done
data: {}
```

### POST `/api/v1/review/pull-request`

Executa review de Pull Request no GitHub com Jira opcional.

#### Request

```
{
  "github_pull_request_url": "https://github.com/org/repo/pull/123",
  "jira_issue_key": "CPJ-123",
  "base_branch": "main",
  "prompt_version": 1
  "model": "openai/gpt-4o-mini"
}
```

#### Response

```
{
  "verdict": "needs_attention",
```

```

    "score": 78,
    "summary": "O PR esta consistente, mas possui alertas relevantes de seguranca e padrao.",
    "pull_request": {
      "owner": "org",
      "repo": "repo",
      "number": 123,
      "title": "Ajusta regras de cobranca",
      "base_branch": "main",
      "head_sha": "abc123",
      "changed_files": 6
    },
    "jira": {
      "issue_key": "CPJ-123",
      "summary": "Ajustar fluxo de renegociacao",
      "criteria count": 3,
      "evaluated": true
    },
    "sections": {
      "code_standard": {
        "status": "warning",
        "findings": []
      },
      "jira_criteria": {
        "status": "passed",
        "criteria": []
      },
      "project_consistency": {
        "status": "passed",
        "findings": []
      },
      "security": {
        "status": "warning",
        "findings": []
      }
    },
    "positives": [
      "Boa separacao por modulo "
    ],
    "recommendations": [
      "Revisar validacoes de entrada sensivel."
    ]
  }
}

```

## POST /api/v1/compliance

Compara tarefa e implementacao.

### Request

```
{
  "task_description": "Permitir renegociacao apenas para contratos ativos e registrar audit",
  "code": "if (contract.active) { renegotiate(contract); audit(contract.id); }",
  "language": "typescript",
  "prompt_version": 1,
  "model": "deepseek/deepseek-v4-flash"
}
```

## Response

```
{
  "compliant": true,
  "compliance_score": 95,
  "covered_requirements": [
    "Valida contrato ativo antes da renegociacao."
  ],
  "missing_requirements": [],
  "partial_requirements": [],
  "verdict": "Aderente."
}
```

## POST `/api/v1/document`

Gera documentacao tecnica ou operacional.

## Request

```
{
  "code": "export function charge(amount: number) { return amount > 0; }",
  "language": "typescript",
  "doc_type": "technical",
  "prompt_version": 1,
  "model": "openai/gpt-4o-mini"
}
```

## Response

```
{
  "doc_type": "technical",
  "title": "Servico de cobranca",
  "description": "Valida se uma cobranca possui valor positivo.",
  "inputs": [
    {
      "name": "amount"
    }
  ]
}
```

```
{
  "type": "number",
  "description": "Valor da cobrança."
},
{
  "outputs": [
    {
      "name": "return",
      "type": "boolean",
      "description": "Indica se o valor é positivo."
    }
  ],
  "side_effects": [],
  "usage_example": "charge(100)",
  "notes": null
}
```

## POST `/api/v1/tests`

Gera casos e arquivo de testes.

### Request

```
{
  "code": "export function charge(amount: number) { return amount > 0; }",
  "language": "typescript",
  "test_framework": "vitest",
  "prompt_version": 1,
  "model": "deepseek/deepseek-v4-flash"
}
```

### Response

```
{
  "framework": "vitest",
  "test_file": "import { expect, it } from 'vitest';\nimport { charge } from './charge';",
  "test_cases": [
    {
      "name": "retorna true para valor positivo",
      "type": "happy_path",
      "description": "Valida a regra principal."
    }
  ],
  "coverage_hints": [
    "Adicionar teste para amount <= 0 quando houver regra de erro."
  ]
}
```



## POST `/api/v1/tests/pull-request`

Gera testes com base nas alteracoes de um Pull Request.

### Request

```
{
  "github_pull_request_url": "https://github.com/org/repo/pull/123",
  "base_branch": "main",
  "test_framework": "vitest",
  "prompt_version": 1,
  "model": "openai/gpt-4o-mini"
}
```

### Response

Usa o mesmo schema de resposta do endpoint `/api/v1/tests`.

## POST `/api/v1/batch`

Executa varios itens em sequencia.

### Request

```
{
  "continue_on_error": true,
  "notify": false,
  "items": [
    {
      "flow_type": "review",
      "payload": {
        "code": "function sum(a, b) { return a + b; }",
        "language": "javascript"
      }
    },
    {
      "flow_type": "tests",
      "payload": {
        "code": "export function charge(amount: number) { return amount > 0; }",
        "language": "typescript",
        "test_framework": "vitest"
      }
    }
  ]
}
```

## Response

```
{
  "batch_id": "uuid-do-batch",
  "status": "success",
  "results": [
    {
      "index": 0,
      "flow_type": "review",
      "execution_id": "execution-review-1",
      "status": "success",
      "cache_hit": false,
      "output": {
        "overall_quality": "good",
        "score": 9,
        "issues": [],
        "positives": [
          "Codigo simples e legivel."
        ],
        "summary": "Sem problemas relevantes."
      },
      "error_message": null
    },
    {
      "index": 1,
      "flow_type": "tests",
      "execution_id": "execution-tests-1",
      "status": "success",
      "cache_hit": true,
      "output": {
        "framework": "vitest",
        "test_file": "import { expect, it } from 'vitest';\n...",
        "test_cases": [],
        "coverage_hints": []
      },
      "error_message": null
    }
  ]
}
```

## Endpoints de historico e analytics

### GET **/api/v1/history**

Lista execucoes recentes.

### Query params

- `limit`
- `cursor`
- `flow_type`
- `status`
- `model`
- `from`
- `to`
- `cache_hit`

## Exemplo

```
GET /api/v1/history?limit=20&flow_type=review&status=success
```

## Response resumida

```
{
  "items": [
    {
      "id": "uuid",
      "type": "review",
      "status": "success",
      "timestamp": "2026-05-26T10:30:00-03:00",
      "duration_ms": 1240,
      "cache_hit": false,
      "source_execution_id": null,
      "telemetry": null,
      "steps": [
        {
          "node_name": "language router",
          "kind": "system",
          "status": "success",
          "duration_ms": 8
        }
      ]
    }
  ],
  "page": {
    "limit": 20,
    "next_cursor": null
  }
}
```

## GET `/api/v1/history/:id`

Retorna o detalhamento completo da execucao, incluindo:

- `input_payload`
- `output_payload`
- `error_message`
- `steps` completos
- `telemetry`

## GET `/api/v1/analytics/usage`

Agrega execucoes e consumo.

### Query params

- `flow_type`
- `model`
- `from`
- `to`

### Response resumida

```
{
  "totals": {
    "executions": 12,
    "successful": 10,
    "failed": 2,
    "cache_hits": 3,
    "prompt_tokens": 14000,
    "completion_tokens": 6200,
    "total_tokens": 20200,
    "cache_read_tokens": 500,
    "cost_total_usd": 0.024,
    "cost_input_usd": 0.009,
    "cost_output_usd": 0.015,
    "average_duration_ms": 980
  },
  "by day": [],
  "by flow": []
}
```

```
"by_model": []
]
```

## Endpoints de prompts

### GET `/api/v1/prompts`

Lista versoes por fluxo. Requer query `flow_type`.

#### Exemplo

```
GET /api/v1/prompts?flow_type=review
```

### GET `/api/v1/prompts/:flowType/active`

Busca a versao ativa de um fluxo.

### GET `/api/v1/prompts/:flowType/:version`

Busca uma versao especifica.

### POST `/api/v1/prompts`

Cria nova versao.

#### Exemplo

```
{
  "flow_type": "document",
  "name": "Document v2",
  "blocks": [
    {
      "block_key": "agent",
      "system_prompt": "Voce gera documentacao tecnica usando o contexto {{language context"
    }
  ]
}
```

### POST `/api/v1/prompts/:flowType/:version/activate`

Ativa uma versao existente.

## Endpoints de modelos

### GET `/api/v1/models`

Lista modelos cadastrados.

### GET `/api/v1/models/default`

Retorna o modelo padrao global.

### POST `/api/v1/models`

#### Request

```
{  
  "name": "anthropic/claude-3.5-haiku"  
}
```

### PATCH `/api/v1/models/:id`

#### Request

```
{  
  "is_default": true  
}
```

### DELETE `/api/v1/models/:id`

Remove um modelo nao padrao.

## Healthcheck e Swagger

### GET `/health`

Retorna o status operacional da API.

## GET `/docs`

Abre a documentacao interativa Swagger/OpenAPI.

## Relacao entre endpoints e persistencia

---

Nem todo endpoint apenas responde e termina. Alguns tambem influenciam o que vai para o historico e para analytics:

- `review`, `compliance`, `document`, `tests` e `pull-request review` podem gerar execucoes persistidas;
- `batch` cria um resumo de lote e pode acionar varios subfluxos persistidos;
- `history` e `analytics` leem a base persistida;
- `prompts` e `models` alteram governanca em runtime.

## Onde validar manualmente

---

Para teste manual rapido, os melhores pontos de entrada sao:

- `requests/cpj-cobranca-api.http`
- `GET /docs`
- `GET /health`

## Relacao com os proximos capitulos

---

Depois deste capitulo, o proximo passo natural e `07-dados-cache-telemetria.md`, que mostra como essas respostas, steps, custos, prompts e modelos ficam guardados no banco.

# Dados, Cache e Telemetria

---

## Objetivo deste capítulo

---

Este capítulo explica como a aplicação persiste execuções, steps, telemetria, prompts, modelos e resumos de batch.

O objetivo é mostrar que a solução não trata o uso de IA como algo efêmero. Cada execução importante deixa rastros que podem ser consultados, agregados e auditados depois.

## Visão geral da persistência

---

O projeto usa PostgreSQL com Prisma e concentra a modelagem principal em:

- `Execution`
- `ExecutionStep`
- `ExecutionTelemetry`
- `PromptVersion`
- `RegisteredModel`
- `GlobalModelSettings`
- `BatchExecution`

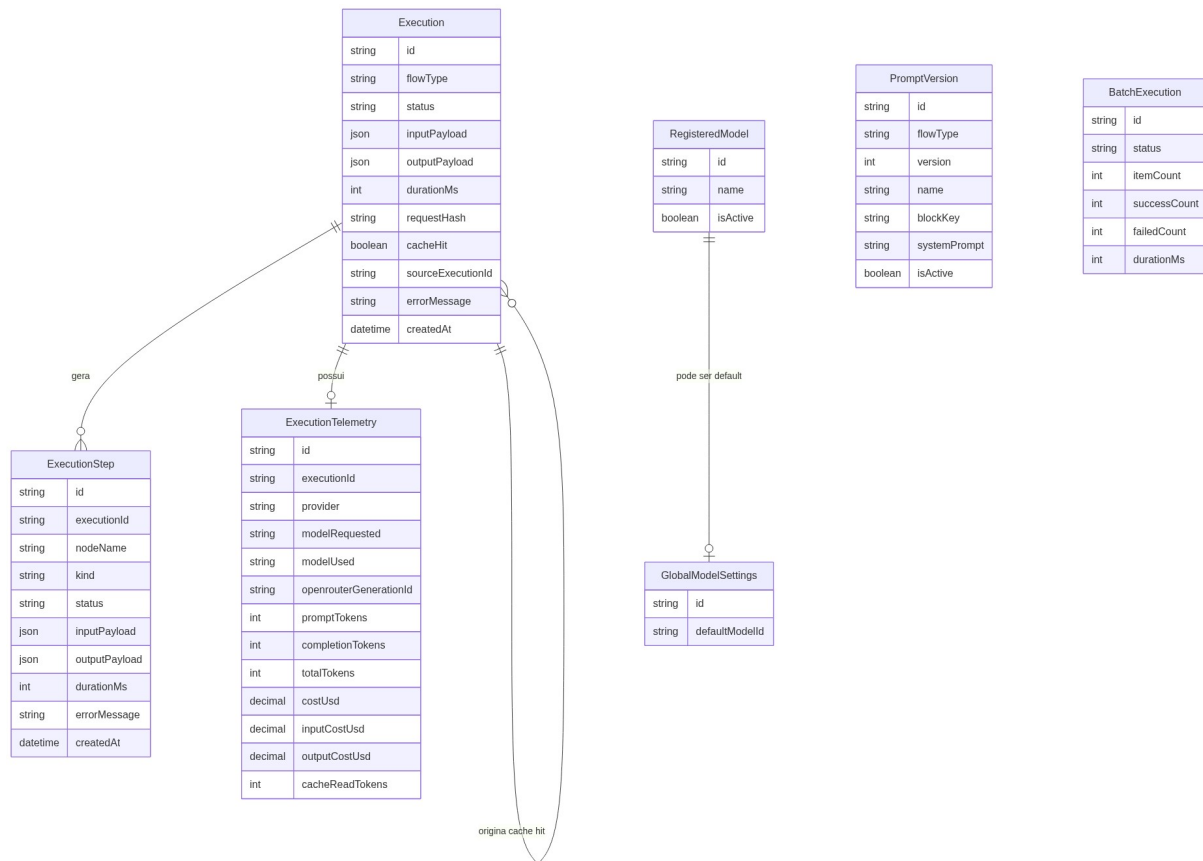
Essa estrutura atende quatro necessidades centrais do caso:

- histórico rastreável;
- governança de prompts;
- governança de modelos;
- medição de custo e uso.

## Diagrama de entidades

---





## Execucoes de fluxo

### Tabela **Execution**

**Execution** é o registro central de uma execucao.

Campos principais:

- **flowType**: qual fluxo rodou;
- **status**: **pending**, **success** ou **failed**;
- **inputPayload**: payload original;
- **outputPayload**: resposta final quando houver sucesso;
- **durationMs**: duracao total;
- **requestHash**: hash usado para cache;
- **cacheHit**: indica reaproveitamento;
- **sourceExecutionId**: aponta para a execucao fonte em caso de cache hit;
- **errorMessage**: mensagem final de falha, quando existir.

## Fluxos registrados

O enum `ExecutionFlowType` cobre:

- `review`
- `compliance`
- `document`
- `tests`
- `batch`
- `pull_request_review`

Isso deixa claro que a persistencia cobre tanto os fluxos obrigatorios quanto alguns diferenciais relevantes.

## Steps de execucao

---

### Tabela `ExecutionStep`

`ExecutionStep` registra o caminho interno de cada execucao.

Campos principais:

- `nodeName`
- `kind`
- `status`
- `inputPayload`
- `outputPayload`
- `durationMs`
- `errorMessage`

### Tipos de step

O enum `ExecutionStepKind` inclui:

- `system`
- `tool`
- `prompt`
- `llm`
- `parser`
- `persistence`
- `webhook`
- `cache`

Na pratica, isso permite diferenciar:

- uma etapa de orquestracao;
- uma tool deterministica;
- uma chamada a agente LLM;
- uma etapa de webhook;
- um cache hit ou cache lookup.

## Telemetria de uso e custo

---

### Tabela `ExecutionTelemetry`

`ExecutionTelemetry` guarda metadados de consumo do provedor LLM.

Campos principais:

- `provider`
- `modelRequested`
- `modelUsed`
- `openrouterGenerationId`
- `promptTokens`
- `completionTokens`
- `totalTokens`

- `costUsd`
- `inputCostUsd`
- `outputCostUsd`
- `cacheReadTokens`

Essa estrutura permite responder perguntas como:

- quanto um fluxo consumiu;
- qual modelo foi usado de fato;
- quanto custou uma execucao;
- quanto do uso veio de leitura de cache do provider.

## Como cache funciona

---

O cache e baseado em hash do payload de entrada.

### Logica geral

1. o engine calcula `requestHash`;
2. procura uma execucao anterior com mesmo hash e status `success`;
3. se encontrar, cria uma nova execucao marcada com `cacheHit=true`;
4. essa nova execucao aponta para a original em `sourceExecutionId`.

### Por que criar nova execucao em cache hit

Essa escolha e importante porque preserva rastreabilidade de uso real.

Sem isso, haveria duas perdas:

- a API devolveria uma resposta sem registrar que aquela chamada aconteceu;
- analytics e historico subestimariam o uso do sistema.

Com o modelo atual, existe reaproveitamento de resultado sem perder trilha operacional.

## Relacao pai-filho no cache

---

A auto-relacao em `Execution` permite ligar:

- uma execucao fonte;
- varias execucoes filhas reaproveitando aquele resultado.

Isso e particularmente util em cenarios de batch, demonstracao repetida ou uso intensivo do mesmo input.

## Historico exposto pela API

---

O historico publico nao retorna o schema Prisma cru. Ele passa por mapeamento para contratos HTTP mais amigaveis.

Exemplos de transformacao:

- `flowType` -> `type`
- `createdAt` -> `timestamp`
- `durationMs` -> `duration_ms`
- `cacheHit` -> `cache_hit`
- `sourceExecutionId` -> `source_execution_id`

O mesmo acontece com telemetria e steps, para manter consistencia com os contratos HTTP da API.

## Filtros e leitura operacional

---

`GET /api/v1/history` permite filtrar por:

- `flow_type`
- `status`
- `model`
- `from`
- `to`

- `cache_hit`

Isso so funciona bem porque a modelagem persistida foi pensada desde o inicio com esses cenarios de consulta em mente.

## Analytics de uso

---

O endpoint `GET /api/v1/analytics/usage` agrega dados a partir das execucoes e da telemetria relacionada.

Os agrupamentos principais sao:

- totais gerais;
- por dia;
- por fluxo;
- por modelo.

Os campos agregados incluem:

- execucoes;
- sucessos e falhas;
- cache hits;
- tokens;
- custos;
- duracao media.

## Prompts versionados no banco

---

### Tabela `PromptVersion`

Em vez de tratar prompt como texto solto em arquivo apenas, a aplicacao pode resolver versoes persistidas por:

- `flowType`

- `version`
- `blockKey`

Cada linha representa um bloco de prompt, não um documento único inteiro.

Isso permite que um fluxo como `review` tenha vários blocos independentes:

- `naming_clarity`
- `error_handling`
- `resource_leak`
- `complexity`
- `security`
- `aggregator`

## Ativação

O campo `isActive` define qual versão está ativa para aquele fluxo em runtime.

## Catálogo de modelos

---

### Tabela `RegisteredModel`

Guarda o catálogo de modelos aceitos pela aplicação.

Campos principais:

- `name`
- `isActive`

### Tabela `GlobalModelSettings`

Guarda a referência do modelo padrão global.

Esse desenho separa duas responsabilidades:

- quais modelos existem e podem ser usados;

- qual deles é o default atual.

## Batch persistido

---

### Tabela `BatchExecution`

Guarda resumo operacional de lotes executados.

Campos principais:

- `status`
- `itemCount`
- `successCount`
- `failedCount`
- `durationMs`

Esse registro não substitui as execuções individuais dos subfluxos. Ele atua como visão consolidada do lote.

## Tipos de status na base

---

### Status de execução

`ExecutionStatus`:

- `pending`
- `success`
- `failed`

### Status de batch

`BatchStatus`:

- `success`



- `partial`
- `failed`

Isso permite representar tanto fluxo individual quanto consolidacao de lote sem forçar um unico enum para cenarios diferentes.

## Como os dados sustentam a governanca do case

---

Essa modelagem da base sustenta varias qualidades importantes da entrega:

- auditoria de como uma resposta foi produzida;
- comparacao entre execucoes;
- medicao de custo do uso de IA;
- ativacao controlada de prompts e modelos;
- reuso por cache sem perder rastreabilidade.

## Limites assumidos no modelo atual

---

O modelo atual foi desenhado para o escopo do case e assume alguns limites:

- nao ha multi-tenant;
- nao ha tabela dedicada de usuarios;
- nao ha fila de jobs persistida;
- `BatchExecution` resume o lote, mas nao modela subitens como entidade propria;
- prompts continuam tendo fallback local em alguns cenarios.

Esses limites nao prejudicam a proposta da entrega, mas deixam claro onde uma versao futura poderia evoluir.

## Relacao com os proximos capitulos

---

Depois deste capitulo, os proximos documentos aprofundam a camada de governanca que usa esses dados:

- [08-prompts-modelos-governanca.md](#)
- [09-painel-web-admin.md](#)
- [11-deploy-operacao.md](#)

# Prompts, Modelos e Governança

---

## Objetivo deste capítulo

---

Este capítulo explica como a aplicação governa o comportamento dos fluxos de IA sem depender exclusivamente de código-fonte hardcoded.

O foco está em dois pilares:

- versionamento e ativação de prompts;
- catálogo e resolução de modelos em runtime.

## Visão geral da governança

---

A governança do projeto foi desenhada para separar duas perguntas:

- **como o agente deve se comportar?**
- **qual modelo deve executar esse comportamento?**

Para responder essas perguntas, a aplicação usa:

- `PromptVersion`, para armazenar blocos de prompt por fluxo;
- `RegisteredModel`, para catalogar modelos aceitos;
- `GlobalModelSettings`, para definir o modelo padrão global;
- `prompt_version` e `model`, para overrides por request.

## Por que essa camada existe

---

Sem uma camada de governança, qualquer ajuste de comportamento exigiria:

- editar arquivo de prompt local;
- rebuild da aplicação;

- novo deploy.

Com a estrutura atual, a API consegue:

- cadastrar novas versoes de prompt;
- ativar uma versao por fluxo;
- trocar o modelo padrao global;
- sobrescrever prompt e modelo em chamadas individuais.

Isso aproxima a entrega de uma operacao real e melhora bastante a avaliacao do case, porque mostra controle fino sobre a camada de IA.

## Versionamento de prompts

---

### Estrutura conceitual

Prompts nao sao tratados como um documento unico por fluxo. Eles sao divididos em blocos menores, o que deixa o comportamento mais modular.

Cada registro de prompt tem:

- `flowType`
- `version`
- `name`
- `blockKey`
- `systemPrompt`
- `isActive`

### Blocos por fluxo

#### Review

O fluxo `review` exige os blocos:

- `naming_clarity`

- `error_handling`
- `resource_leak`
- `complexity`
- `security`
- `aggregator`

## Compliance, document e tests

Esses fluxos exigem apenas:

- `agent`

## Pull request review

O fluxo `pull_request_review` exige:

- `code_standard`
- `jira_criteria`
- `project_consistency`
- `security`
- `aggregator`

## Ativacao de versoes

Para cada fluxo, a aplicacao usa uma versao ativa por padrao. Essa ativacao e controlada pelo endpoint:

```
POST /api/v1/prompts/:flowtype/:version/activate
```

Quando uma nova versao vira ativa:

- requests futuras passam a usar esse conjunto;
- nao e necessario rebuild;

- o comportamento do fluxo muda de forma rastreável.

## Override por request

---

Mesmo com uma versão ativa global por fluxo, a API aceita `prompt_version` em requests individuais.

Isso permite:

- comparar duas versões sem trocar o padrão global;
- testar comportamento novo em ambiente controlado;
- validar ajustes antes de promover uma versão.

## Resolver de prompts em runtime

---

A resolução de prompts é feita pela camada `PromptRuntimeResolver`.

Ela é responsável por:

- carregar a versão ativa quando `prompt_version` não é informado;
- carregar a versão específica quando houver override;
- validar se todos os blocos obrigatórios daquele fluxo existem;
- montar um runtime prompt set compatível com o fluxo.

## Fallback legado

---

O projeto ainda possui um fallback local por meio de `LegacyPromptRuntimeResolver`.

Isso significa que, na ausência de repositório persistido configurado, o sistema consegue continuar operando com catálogos locais de prompt.

Esse comportamento foi uma escolha pragmática para:

- manter testes simples;

- não bloquear ambientes sem banco configurado;
- preservar backward compatibility durante a evolução da base.

## Seed inicial de prompts

---

O `prisma/seed.ts` cria o conjunto inicial de prompts quando a tabela ainda não possui dados.

O seed inclui:

- `Review v1`
- `Compliance v1`
- `Document v1`
- `Tests v1`
- `Pull Request Review v1`

Isso garante que o projeto sobe com um conjunto funcional de comportamento sem exigir cadastro manual logo no primeiro boot.

## Catalogo de modelos

---

### Tabela `RegisteredModel`

O catalogo de modelos define quais nomes podem ser usados pela API.

Cada modelo possui:

- `id`
- `name`
- `isActive`

Isso evita que qualquer request informe nomes arbitrários ou inconsistentes.

### Tabela `GlobalModelSettings`

Essa tabela guarda o modelo padrao global da aplicacao.

Com isso, a escolha do modelo default deixa de ser apenas uma env var e passa a ser parte da governanca persistida do sistema.

## Modelo padrao global

---

Quando um endpoint nao recebe o campo `model`, a aplicacao tenta usar:

1. o modelo padrao global persistido;
2. o fallback operacional definido por `OPENROUTER_DEFAULT_MODEL`, quando necessario.

Esse desenho equilibra dois objetivos:

- governanca persistida e editavel por API;
- resiliencia em ambientes onde a tabela ainda nao esteja pronta.

## Seed inicial de modelos

---

O seed inicial cadastra:

- `openai/gpt-4o-mini`
- `deepseek/deepseek-v4-flash`

E define como padrao inicial:

- `openai/gpt-4o-mini`

## Override de modelo por request

---

Assim como acontece com prompts, requests individuais podem informar `model`.

Esse override so funciona se:

- o modelo estiver cadastrado;



- o modelo estiver ativo.

Com isso, a API permite experimentação controlada sem abrir mão de governança.

## Endpoints de governança

---

### Prompts

- `GET /api/v1/prompts`
- `GET /api/v1/prompts/:flowType/active`
- `GET /api/v1/prompts/:flowType/:version`
- `POST /api/v1/prompts`
- `POST /api/v1/prompts/:flowType/:version/activate`

### Modelos

- `GET /api/v1/models`
- `GET /api/v1/models/default`
- `POST /api/v1/models`
- `PATCH /api/v1/models/:id`
- `DELETE /api/v1/models/:id`

## Relação com os fluxos

---

Essa camada não é isolada do restante da aplicação. Ela participa diretamente da execução dos fluxos:

- services resolvem modelo em runtime;
- engines resolvem prompt set antes de invocar o grafo;
- PR review e review tradicional usam blocos diferentes;
- o painel web usa esses endpoints para operação administrativa.

## Benefícios técnicos da governança atual

---

Os principais ganhos desse desenho são:

- menor acoplamento entre comportamento de IA e deploy;
- possibilidade de experimentação controlada;
- rastreabilidade de mudanças de prompt;
- controle sobre quais modelos podem ser usados;
- base clara para auditoria e administração.

## Limites assumidos

---

O modelo atual ainda tem alguns limites conscientes:

- não existe histórico de quem ativou uma versão;
- não há aprovação em duas etapas para mudança de prompt ou modelo;
- prompts não possuem ambiente separado por tenant ou workspace;
- não existe avaliação automática de performance de prompt por versão.

Mesmo assim, para o escopo do case, a governança entregue já mostra maturidade acima do mínimo exigido.

## Relação com os próximos capítulos

---

Depois deste capítulo, a leitura natural é:

- [09-painel-web-admin.md](#), para ver essa governança operada via interface;
- [10-como-rodar.md](#), para subir tudo localmente;
- [13-extensibilidade.md](#), para entender como adicionar novos blocos, fluxos e modelos.

# Painel Web Admin

---

## Objetivo deste capítulo

---

Este capítulo descreve o painel web localizado em `apps/web`, que funciona como uma camada visual de operacao sobre a API.

O painel nao substitui a API nem concentra a logica principal dos fluxos. Ele atua como console tecnico para:

- visualizar metricas;
- consultar historico;
- administrar prompts e modelos;
- executar fluxos de forma guiada;
- validar status da API e acesso ao Swagger.

## Papel do painel no contexto do case

---

O case nao exigia um frontend administrativo, mas essa interface ajuda muito em avaliacao e demonstracao.

Ela torna mais facil:

- explorar os recursos sem depender apenas de JSON bruto;
- inspecionar dados persistidos;
- provar que analytics, historico, prompts e modelos realmente funcionam;
- operar fluxos principais com menos atrito.

## Stack do painel

---

O painel foi construido com:

- Next.js 16

- React 19
- Ant Design 6
- Ant Design Charts

Essa escolha segue a ideia de uma interface operacional, densa e objetiva, em vez de uma camada visual decorativa ou de marketing.

## Integracao com a API

---

Toda a informacao do painel vem da API. Ele usa `NEXT_PUBLIC_API_BASE_URL` para definir o endpoint base em runtime.

Isso preserva uma separacao clara:

- a API continua sendo a fonte de verdade;
- o painel e apenas cliente da API.

## Navegacao principal

---

O shell principal expoe as seguintes rotas:

- `/` - Dashboard
- `/history` - Historico
- `/prompts` - Prompts
- `/models` - Modelos
- `/execute` - Executar fluxos
- `/status` - Status/API Docs

## Dashboard

---

O dashboard e a visao inicial do painel. Ele se apoia em `GET /api/v1/analytics/usage`.

### Informacoes exibidas

- total de execucoes;
- custo total;
- taxa de cache hit;
- taxa de sucesso e falha;
- tempo medio;
- ranking por fluxo;
- ranking por modelo;
- pulso diario de tokens.

## Objetivo operacional

O dashboard transforma a telemetria persistida em uma leitura rapida da saude do sistema. E a tela mais util para perceber padroes de uso e custo sem abrir execucoes individuais.

## Historico

---

A tela de historico consome:

- `GET /api/v1/history`
- `GET /api/v1/history/:id`

## Recursos principais

- filtro por fluxo;
- filtro por status;
- paginacao via cursor;
- abertura de drawer com detalhe da execucao;
- exibicao de telemetria;
- timeline de steps;
- visualizacao de payloads de entrada e saida.

## Valor tecnico

Essa pagina mostra com clareza que a persistencia nao e apenas conceitual. O avaliador consegue ver:

- execucoes reais;
- steps reais;
- custos e tokens;
- payloads associados.

## Prompts

---

A tela de prompts opera a camada de governanca de comportamento.

### Recursos principais

- escolha do fluxo;
- carregamento da versao ativa;
- visualizacao de versoes existentes;
- criacao de nova versao com blocos por fluxo;
- ativacao de versao.

### Diferenca por fluxo

A interface se adapta ao tipo de fluxo:

- fluxos simples usam apenas **agent**;
- **review** expoe todos os blocos especialistas e aggregator;
- **pull\_request\_review** expoe os blocos especificos desse fluxo.

## Modelos

---

A tela de modelos opera o catalogo permitido da API.

### Recursos principais

- cadastro de novo modelo;
- listagem de modelos;
- ativacao/desativacao;
- definicao de modelo padrao;
- exclusao de modelo nao padrao.

## Valor para o case

Essa tela ajuda a demonstrar que a escolha do modelo nao esta enterrada em env ou no codigo do runner. Existe de fato uma camada administrativa por cima.

## Executar fluxos

---

A tela **Executar fluxos** e uma camada guiada para testes manuais.

### Abas disponiveis

- Review
- Compliance
- Document
- Tests
- PR Review
- PR Tests
- Batch

### Objetivo

Essa pagina permite executar os fluxos mais importantes sem montar requests manualmente fora da aplicacao.

Ela e especialmente util para:

- demonstracao do case;
- validacao manual rapida;

- exploracao dos contratos por quem ainda nao viu o `.http`.

## Status/API Docs

---

A tela de status tem foco em conectividade e operacao.

### Recursos

- consulta ao `GET /health`;
- exibicao do estado operacional;
- link para Swagger;
- link para o JSON OpenAPI.

Essa pagina funciona como ponto rapido para confirmar se a API esta disponivel e para abrir a documentacao interativa.

## Cliente HTTP compartilhado

---

O painel centraliza chamadas HTTP em uma camada de cliente comum.

Isso ajuda a:

- manter endpoints concentrados em um lugar;
- padronizar serializacao de query string;
- manter tipos alinhados entre frontend e API.

## Experiencia esperada do avaliador

---

Com o painel ativo, um avaliador tecnico consegue:

1. abrir o dashboard e ver o resumo operacional;
2. consultar historico e detalhes de execucao;
3. alterar prompts ou modelos;



4. disparar fluxos manualmente;
5. abrir Swagger e cruzar contratos.

Isso melhora bastante a experiencia de entendimento do projeto.

## Limites assumidos no painel atual

---

O painel foi desenhado como console tecnico e assume alguns limites:

- nao ha autenticacao;
- nao ha controle de permissao por perfil;
- nao existe editor rico de prompt;
- o dashboard usa leitura agregada simples, nao drill-down analitico profundo;
- o execute page nao expoe todos os campos possiveis de todos os endpoints.

Esses limites sao coerentes com o escopo do case e nao diminuem o valor operacional da interface entregue.

## Relacao com os proximos capitulos

---

Depois deste capitulo, o passo natural e [10-como-rodar.md](#), para subir API, banco e painel juntos em ambiente local.

# Como Rodar

## Objetivo deste capítulo

Este capítulo mostra como executar o projeto localmente, com e sem Docker, e como validar rapidamente que a aplicação subiu corretamente.

## Pre-requisitos

Para rodar o projeto localmente, o ambiente esperado inclui:

- Node.js 22 ou superior;
- npm;
- Docker e Docker Compose;
- PostgreSQL local, caso opte por rodar sem container para o banco.

## Instalar dependencias

```
npm install
```

O `postinstall` do projeto executa `prisma generate`, então o Prisma Client fica preparado logo na instalação.

## Variáveis de ambiente

O ponto de partida é `.env.example`.

```
cp .env.example .env
```

## Variáveis mais importantes

- `DATABASE_URL`

- `OPENROUTER_API_KEY`
- `OPENROUTER_DEFAULT_MODEL`
- `EXTERNAL_RETRY_ATTEMPTS`
- `EXTERNAL_RETRY_BASE_DELAY_MS`
- `NEXT_PUBLIC_API_BASE_URL`

## Variaveis opcionais de integracao

- `GITHUB_TOKEN`
- `JIRA_BASE_URL`
- `JIRA_EMAIL`
- `JIRA_API_TOKEN`
- `LANGSMITH_TRACING`
- `LANGSMITH_API_KEY`
- `LANGSMITH_PROJECT`
- `WEBHOOK_CALLBACK_URL`

## Rodando em desenvolvimento local

---

### 1. Subir PostgreSQL

Se quiser usar apenas o banco em container:

```
docker compose up -d postgres
```

### 2. Ajustar `.env`

Em ambiente local sem a API em container, a `DATABASE_URL` padrao deve apontar para `localhost`.

### 3. Rodar migrations e seed

```
npx prisma migrate dev --name init
npx prisma db seed
```

Ou, se preferir os scripts do projeto:

```
npm run prisma:migrate
npm run prisma:seed
```

## 4. Iniciar a API

```
npm run dev
```

## 5. Iniciar o painel web

Em outro terminal:

```
npm run dev:web
```

## URLs locais esperadas

---

Com tudo no ar:

- API: <http://localhost:3000>
- Health: <http://localhost:3000/health>
- Swagger: <http://localhost:3000/docs>
- Painel: <http://localhost:3001>

## Rodando com Docker Compose

---

Para subir API, banco e painel juntos:

```
cp .env.example .env
docker compose up --build -d
```

## Observacoes importantes

- em Docker, a API usa `postgres` como host de banco;
- a API executa `prisma migrate deploy` e `prisma db seed` antes de subir;
- o painel sobe apontando para `http://localhost:3000` por padrao.

## Logs uteis

---

### API

```
docker compose logs -f api
```

### Painel web

```
docker compose logs -f web
```

### Banco

```
docker compose logs -f postgres
```

## Validacao rapida de subida

---

### Healthcheck

```
curl http://localhost:3000/health
```

Resposta esperada:

```
{  
  "status": "ok"  
}
```

### Swagger

Abrir:

```
http://localhost:3000/docs
```

## Painel

Abrir:

```
http://localhost:3001
```

## Validacao manual dos fluxos

---

O caminho mais rapido para validar os endpoints e:

- abrir `requests/cpj-cobranca-api.http`;
- executar os exemplos principais;
- conferir o historico em `/api/v1/history`;
- abrir o painel e validar dashboard, prompts, modelos e execute page.

## Ordem recomendada de verificacao

---

1. `GET /health`
2. `GET /docs`
3. `POST /api/v1/review`
4. `GET /api/v1/history`
5. painel em `http://localhost:3001`

## Rodando somente a web

---

Se a API ja estiver no ar em outro endereco, basta ajustar:

```
NEXT_PUBLIC_API_BASE_URL=http://localhost:3000
```

E depois subir a web:

```
npm run dev:web
```

## Rodando build de producao localmente

### API

```
npm run build
npm start
```

### Web

```
npm run build:web
npm run start --workspace apps/web
```

## Problemas comuns logo na subida

- `OPENROUTER_API_KEY` vazia: fluxos com LLM nao executam corretamente;
- `DATABASE_URL` errada: migrations e API falham ao conectar;
- banco ainda nao pronto: a API pode falhar em boot se a dependencia nao estiver acessivel;
- `NEXT_PUBLIC_API_BASE_URL` incorreta: o painel sobe, mas nao consegue carregar dados;
- `GITHUB_TOKEN` ausente: PR review e PR tests podem falhar em repositorios que exigem autenticacao.

## Comandos de desenvolvimento mais usados

```
npm run dev
npm run dev:web
npm test
npm run typecheck
npm run lint
npm run build
```

```
npm run build:web
```

## Relacao com os proximos capitulos

---

Depois deste capitulo, a leitura natural e:

- [11-deploy-operacao.md](#)
- [12-testes-validacao.md](#)
- [14-troubleshooting-faq.md](#)



# Deploy e Operacao

## Objetivo deste capitulo

Este capitulo descreve como o projeto esta preparado para deploy e operacao em producao, com foco no fluxo atual baseado em Docker Compose, GitHub Actions, VPS e Nginx.

## Visao geral do deploy

O deploy de producao foi desenhado para funcionar assim:

1. push na branch `main`;
2. GitHub Actions roda CI;
3. workflow gera `.env` de producao;
4. workflow envia configuracao para a VPS;
5. script remoto atualiza o codigo e sobe os containers;
6. healthchecks validam API e web.



## Workflow GitHub Actions

O workflow principal esta em:

```
github/workflows/deploy.yml
```

### Gatilhos

- `push` para `main`
- `workflow_dispatch`

### Etapas de CI

Antes do deploy, o pipeline executa:

- `npm ci`
- `npm run lint`
- `npm run build`
- `npm run build:web`

Isso ajuda a impedir deploy de código quebrado logo na etapa de integração.

## Geracao do `.env` de producao

---

O workflow monta um `.env.production` temporario com variaveis como:

- `OPENROUTER_API_KEY`
- `OPENROUTER_DEFAULT_MODEL`
- `EXTERNAL_RETRY_ATTEMPTS`
- `EXTERNAL_RETRY_BASE_DELAY_MS`
- `GITHUB_TOKEN`
- `JIRA_BASE_URL`
- `JIRA_EMAIL`
- `JIRA_API_TOKEN`
- `LANGSMITH_TRACING`
- `LANGSMITH_API_KEY`
- `WEBHOOK_CALLBACK_URL`
- `API_PORT_MAP`
- `WEB_PORT_MAP`

Esse arquivo é enviado para a VPS e copiado como `.env` do projeto.

## Docker Compose em producao

---

O deploy usa:

- `docker-compose.yml`
- `docker-compose.prod.yml`

O compose base define a estrutura geral dos serviços. O override de produção ajusta principalmente:

- secrets e envs reais;
- portas bindadas em `127.0.0.1`;
- configuração de banco em produção;
- `NEXT_PUBLIC_API_BASE_URL` para o domínio esperado.

## Serviços principais

---

### postgres

Banco relacional do projeto.

### api

Serviço Fastify com Prisma, fluxos de IA, Swagger e endpoints administrativos.

### web

Painel Next.js consumindo a API já publicada.

## Script remoto de deploy

---

O orquestrador remoto está em:

```
scripts/deploy.sh
```

### Responsabilidades do script

- entrar no diretório de deploy;

- fazer `git fetch`, `checkout` e `pull`;
- instalar configs de Nginx se necessario;
- recarregar Nginx;
- executar `docker compose up -d --build --remove-orphans`;
- esperar o banco;
- validar healthchecks da API e da web.

## Healthchecks de producao

---

O script usa:

- API: `http://127.0.0.1:3020/docs`
- Web: `http://127.0.0.1:3021`

Isso prova que os servicos subiram localmente na VPS antes da exposicao via Nginx.

## Nginx

---

O deploy assume arquivos de configuracao em:

```
deploy/nginx/
```

No primeiro deploy, o script copia configs para:

- `/etc/nginx/sites-available`
- `/etc/nginx/sites-enabled`

Depois valida com `nginx -t` e recarrega o servico.

## Segredos e variaveis esperadas

---

### Secrets do GitHub Actions

Os principais secrets/vars usados no deploy são:

- `VPS_SSH_KEY`
- `VPS_HOST`
- `VPS_PORT`
- `VPS_USER`
- `OPENROUTER_API_KEY`
- `OPENROUTER_DEFAULT_MODEL`
- `EXTERNAL_RETRY_ATTEMPTS`
- `EXTERNAL_RETRY_BASE_DELAY_MS`
- `GH_PAT`
- `JIRA_BASE_URL`
- `JIRA_EMAIL`
- `JIRA_API_TOKEN`
- `LANGSMITH_TRACING`
- `LANGSMITH_API_KEY`
- `WEBHOOK_CALLBACK_URL`

## Variáveis de infraestrutura

- `DEPLOY_PATH`
- `REPO_URL`

## Comportamento esperado do deploy

---

Quando o fluxo está correto:

1. o workflow valida build;
2. o servidor recebe a nova `.env`;
3. o repositório remoto é atualizado na VPS;
4. os containers são rebuildados;

5. healthchecks ficam verdes;
6. o ambiente publicado passa a servir a nova versao.

## Operacao no dia a dia

---

As operacoes mais comuns em producao tendem a ser:

- acompanhar logs dos containers;
- validar healthcheck;
- revisar erro de integracao externa;
- confirmar se prompts/modelos no banco batem com o esperado;
- acompanhar custo e tokens via historico e analytics.

## Logs uteis em ambiente remoto

---

No host da VPS, comandos uteis incluem:

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml logs -f api
docker compose -f docker-compose.yml -f docker-compose.prod.yml logs -f web
docker compose -f docker-compose.yml -f docker-compose.prod.yml logs -f postgres
```

## Pontos de atencao operacionais

---

- `OPENROUTER_API_KEY` ausente ou invalida interrompe os fluxos com LLM;
- `GH_PAT` ausente pode afetar PR review e PR tests;
- `LANGSMITH_TRACING=true` sem chave util nao ajuda na observabilidade;
- `WEBHOOK_CALLBACK_URL` com erro nao quebra o fluxo principal, mas gera step de falha no historico;
- valores ruins de retry podem aumentar latencia em caso de erro externo.

## Recuperacao e diagnostico

---

Quando algo falha em producao, a ordem mais util de diagnostico costuma ser:

1. conferir execucao do workflow;
2. validar `.env` gerado;
3. revisar logs do `api`;
4. testar `GET /health` e `GET /docs`;
5. revisar conectividade com banco e OpenRouter;
6. validar Nginx e portas locais.

## Limites assumidos na operacao atual

---

O deploy atual foi desenhado para o case e, por isso, assume:

- uma VPS unica;
- deploy por Docker Compose;
- ausencia de orquestrador como Kubernetes;
- observabilidade operacional relativamente simples;
- rollout direto, sem estrategia blue/green.

Esses limites sao coerentes com o escopo da entrega e deixam a operacao mais facil de entender.

## Relacao com os proximos capitulos

---

Depois deste capitulo, a leitura natural e:

- [12-testes-validacao.md](#)
- [14-troubleshooting-faq.md](#)

# Testes e Validacao

---

## Objetivo deste capitulo

---

Este capitulo mostra como validar tecnicamente a entrega: testes automatizados, comandos de qualidade e validacoes manuais mais relevantes.

## Filosofia de validacao do case

---

Como este projeto foi construido para um case tecnico, a validacao precisa provar tres coisas:

- a API sobe;
- os fluxos principais respondem corretamente;
- a base tecnica e consistente o suficiente para evolucao.

Por isso, a estrategia combina:

- testes automatizados;
- verificacoes de build e lint;
- validacao manual de endpoints e painel.

## Suite automatizada do backend

---

O backend usa Vitest como runner principal.

## Cobertura funcional esperada

A suite cobre, entre outros pontos:

- rotas principais;
- contracts Zod;
- historico;
- analytics;



- prompts;
- modelos;
- review;
- compliance;
- document;
- tests;
- batch;
- PR review;
- PR tests;
- retry;
- docker e workflow em pontos criticos.

## Suite automatizada da web

---

O frontend em `apps/web` tambem possui testes com Vitest e Testing Library.

Esses testes validam paginas como:

- dashboard;
- history;
- prompts;
- models;
- execute;
- status;
- cliente HTTP compartilhado.

## Comandos principais de validacao

---

### Backend

```
npm test
npm run typecheck
npm run lint
npm run build
```

## Frontend

```
npm run build:web
npm run test:web
```

## O que cada comando prova

---

### `npm test`

Prova que a suite principal do backend segue verde.

### `npm run typecheck`

Prova que o TypeScript compila semanticamente sem erros.

### `npm run lint`

Prova consistencia estatica e higiene de codigo nas pastas alvo.

### `npm run build`

Prova que a API empacota corretamente para producao.

### `npm run build:web`

Prova que o painel compila e gera build de producao.

### `npm run test:web`

Prova que a camada de interface principal continua funcional.

## Validacao manual recomendada

---

Mesmo com testes automatizados, o case ganha muito quando passa por uma rodada manual simples.

## Passo 1

Validar:

```
GET /health
```

## Passo 2

Abrir:

```
GET /docs
```

## Passo 3

Executar pelo menos uma request de cada fluxo principal:

- `review`
- `compliance`
- `document`
- `tests`

## Passo 4

Executar:

- `batch`
- `review/pull-request`
- `tests/pull-request`

quando as credenciais externas estiverem disponiveis.

## Passo 5

Conferir:

- `GET /api/v1/history`
- `GET /api/v1/analytics/usage`

## Passo 6

Abrir o painel e validar:

- dashboard;
- history;
- prompts;
- models;
- execute;
- status.

## Validacao via arquivo `.http`

---

O arquivo:

```
requests/cpj-cobranca-api.http
```

e a referencia mais pratica para demonstracao manual dos endpoints.

Ele concentra exemplos prontos para uso durante:

- desenvolvimento;
- depuracao;
- demonstracao;
- avaliacao do case.

## Critérios de aceite técnicos

---

Uma leitura objetiva dos criterios de aceite para a entrega atual seria:

1. a API sobe localmente;
2. o banco conecta e persiste execucoes;
3. os quatro fluxos obrigatorios funcionam;
4. o historico pode ser consultado;
5. a documentacao OpenAPI esta disponivel;
6. o projeto roda em Docker;
7. o painel web consegue consumir a API;
8. os diferenciais mais importantes estao operacionais.

## O que esta sendo efetivamente provado

---

Ao combinar testes, build e validacao manual, a entrega demonstra:

- corretude contratual;
- funcionamento dos endpoints;
- integridade da base de codigo;
- consistencia de build;
- prontidao para avaliacao tecnica.

## Testes versus integracoes externas

---

Nem tudo precisa depender de chamadas reais a OpenRouter, GitHub ou Jira para ser validado com confianca.

A estrategia do projeto privilegia:

- testes de contratos e rotas;
- isolamento de dependencias;
- validacao estrutural da aplicacao;
- demonstracao manual quando integracoes reais estiverem disponiveis.

Isso é adequado ao contexto de case, onde a clareza do desenho conta tanto quanto a presença de todos os ambientes externos.

## Riscos residuais

---

Mesmo com a suite atual, alguns riscos continuam existindo:

- variação de qualidade do LLM em execução real;
- instabilidade de credenciais externas;
- comportamento de custo/token dependente do provedor;
- falhas de rede em PR review, PR tests ou webhook.

Esses pontos, no entanto, ficam bem delimitados e visíveis pela arquitetura e pela telemetria do projeto.

## Relação com os próximos capítulos

---

Depois deste capítulo, os capítulos mais úteis para aprofundar a manutenção do projeto são:

- [13-extensibilidade.md](#)
- [14-troubleshooting-faq.md](#)

# Extensibilidade

---

## Objetivo deste capítulo

---

Este capítulo explica como evoluir a solução sem quebrar o desenho atual. O foco é mostrar como adicionar novos fluxos, linguagens, agentes, tools, prompts e modelos com o menor atrito possível.

## Princípio de extensibilidade da base

---

O projeto foi estruturado para crescer por composição de módulos, e não por acúmulo de lógica em um único ponto central.

Na prática, isso significa que evoluções costumam seguir um padrão claro:

- criar contrato;
- criar módulo ou expandir módulo existente;
- registrar rota;
- plugar persistência quando necessário;
- incluir prompt e governança;
- adicionar testes.

## Como adicionar um novo fluxo

---

O caminho natural para um novo fluxo seria:

1. criar schema Zod em `src/shared/contracts`;
2. criar módulo em `src/modules/<novo-fluxo>`;
3. adicionar `routes`, `controllers`, `services`, `engines` e `graphs` conforme a complexidade;
4. registrar a rota em `src/app.ts`;
5. decidir se o fluxo deve persistir execução;

6. criar documentacao OpenAPI;
7. adicionar exemplos no `.http`;
8. adicionar testes.

## Como adicionar uma nova linguagem ao review

---

O fluxo `review` foi desenhado para suportar evolucao por linguagem.

Para incluir uma nova linguagem, o caminho geral seria:

1. atualizar o schema de linguagens suportadas;
2. criar um novo `language profile`;
3. criar um novo grafo especifico da linguagem;
4. atualizar o `LanguageRouter`;
5. registrar o novo grafo no `ReviewFlowGraph`;
6. cobrir com testes.

Esse desenho evita condicoes gigantes espalhadas em varios lugares.

## Como adicionar um novo agente especialista

---

No review, especialistas sao blocos bem definidos.

Para criar um novo especialista:

1. definir o objetivo da analise;
2. criar a classe do agente;
3. adicionar bloco de prompt correspondente;
4. incluir o agente no grafo de linguagem;
5. decidir como o aggregator deve consumir a nova saida;
6. atualizar contratos, se necessario;
7. adicionar testes.



## Como adicionar uma nova tool determinística

---

As tools são uma boa porta de extensão quando o comportamento pode ser expresso por heurística objetiva.

O caminho geral seria:

1. criar ou expandir o runner determinístico do fluxo;
2. devolver achados em formato compatível com o contexto do agente;
3. registrar o step no grafo;
4. ajustar prompts se o agente passar a consumir esse novo sinal.

## Como adicionar um novo bloco de prompt

---

Quando um fluxo precisa de governança mais fina, um novo bloco pode ser uma boa evolução.

O processo natural seria:

1. incluir o novo `PromptBlockKey`;
2. ajustar schema e validações da camada de prompts;
3. atualizar seed, se fizer sentido;
4. ajustar o resolver de runtime;
5. adaptar o fluxo que vai consumir esse bloco;
6. refletir a mudança no painel de prompts.

## Como adicionar um novo modelo

---

Se o objetivo for apenas experimentar um novo modelo já suportado pelo OpenRouter, o caminho mais simples e administrativo:

1. cadastrar em `/api/v1/models`;
2. ativar ou definir como padrão;
3. testar via override por request ou como default global.

Se a evolucao exigir outro provider, o impacto e maior e tende a envolver:

- camada de factory do chat model;
- telemetria;
- retry;
- shape de custo e metadata.

## Como evoluir PR review

---

O fluxo `pull_request_review` ja esta desenhado para crescer por secoes.

Alguns exemplos de evolucao natural:

- novo agente de performance;
- novo agente de observabilidade;
- leitura de mais fontes de contexto;
- ampliacao da carga de padroes do projeto;
- novas secoes no aggregator final.

## Como evoluir PR tests

---

O fluxo de testes por PR pode crescer em especial nos pontos:

- heurísticas de extracao de funcoes criticas;
- suporte mais forte a classes e metodos;
- enriquecimento do contexto do diff;
- estrategias especificas por framework.

## Como evoluir historico e analytics

---

A modelagem atual ja oferece boa base para:

- novos filtros;

- novos agrupamentos;
- dashboards mais ricos;
- comparacao entre modelos;
- comparacao entre versoes de prompt.

Em outras palavras: a base ja esta preparada para evoluir de observacao operacional simples para analise mais profunda.

## Como evoluir o painel web

---

O painel tambem foi separado por features, o que favorece crescimento por tela.

Evolucoes naturais incluem:

- filtros mais ricos no dashboard;
- visualizacao lado a lado de prompts;
- editor mais avancado de blocos;
- telas especificas para PR review e PR tests;
- comparador de custo por modelo.

## Boas praticas para evoluir esta base

---

As melhores evolucoes tendem a seguir alguns principios:

- manter contratos e implementacao alinhados;
- preferir extensao modular a refatoracoes amplas desnecessarias;
- preservar rastreabilidade e steps;
- adicionar testes junto com novas capacidades;
- documentar cada novo comportamento em README, Swagger e docs.

## O que nao vale a pena fazer cedo demais

---

Algumas evolucoes podem parecer atraentes, mas nao sao prioridade imediata para esta base:

- microservicacao precoce;
- excesso de abstracoes para todos os fluxos;
- provider-agnostic generico demais antes de necessidade real;
- fila distribuida complexa sem demanda operacional concreta.

## Relacao com os proximos capitulos

---

Depois deste capitulo, o capitulo mais util para sustentar a manutencao do projeto no dia a dia e [14-troubleshooting-faq.md](#).

# Troubleshooting e FAQ

---

## Objetivo deste capítulo

---

Este capítulo reúne problemas comuns, sintomas prováveis e caminhos de diagnóstico para API, banco, painel, integrações externas e deploy.

## A API não sobe

---

### Sintomas comuns

- erro logo no boot;
- `GET /health` indisponível;
- container `api` reiniciando.

### Verificações recomendadas

1. conferir `DATABASE_URL`;
2. conferir `OPENROUTER_API_KEY`;
3. validar se o banco está acessível;
4. revisar logs da API;
5. validar se migrations e seed foram aplicados.

## Erro de banco de dados

---

### Causas comuns

- host incorreto na `DATABASE_URL`;
- banco ainda não pronto;
- porta errada;
- migrations não aplicadas.

## Acoes uteis

```
docker compose logs -f postgres
npx prisma migrate dev
npm run prisma:deploy
```

## GET /health responde, mas fluxos falham

---

Isso normalmente indica que a API subiu, mas alguma integracao essencial nao esta funcional.

### Verificar

- `OPENROUTER_API_KEY`
- modelo padrao global
- modelo solicitado na request
- conectividade externa
- retry e mensagens de erro no historico

## Falha com OpenRouter

---

### Causas provaveis

- chave invalida;
- limite do provider;
- modelo nao disponivel;
- telemetria externa indisponivel.

### Observacoes

- o fluxo de retry pode repetir chamadas transitorias;
- telemetria adicional nao impede necessariamente a resposta principal;

- o historico pode mostrar falha em step LLM com detalhes uteis.

## O modelo informado na request nao funciona

---

### Causas provaveis

- modelo nao cadastrado;
- modelo cadastrado, mas inativo;
- typo no nome enviado.

### O que validar

- `GET /api/v1/models`
- modelo padrao em `GET /api/v1/models/default`
- payload enviado no endpoint

## Prompt nao muda mesmo depois de cadastrar nova versao

---

### Causas comuns

- nova versao criada, mas nao ativada;
- request usando `prompt_version` antigo explicitamente;
- fluxo com fallback local em ambiente sem repositorio persistido ativo.

### Verificar

- `GET /api/v1/prompts/:flowType/active`
- `POST /api/v1/prompts/:flowType/:version/activate`
- payload efetivo da chamada

## PR review ou PR tests falham

---

### Causas prováveis

- `GITHUB_TOKEN` ausente ou insuficiente;
- URL do PR inválida;
- repositório privado sem permissão;
- `JIRA_*` ausentes quando Jira é exigido;
- indisponibilidade de GitHub ou Jira.

### Verificações úteis

- confirmar formato da URL do PR;
- revisar se `base_branch` faz sentido;
- validar se `jira_issue_key` é realmente necessário;
- checar logs e steps como `github_fetch` e `jira_fetch`.

## Webhook falha

---

### Comportamento esperado

Falha no callback de webhook não derruba a resposta principal do fluxo.

### Onde aparece

- em `ExecutionStep`, como `webhook_callback` com status `failed`;
- em logs da API;
- potencialmente no histórico detalhado.

### O que revisar

- `WEBHOOK_CALLBACK_URL`



- disponibilidade do endpoint externo;
- autenticação exigida no destino;
- latência e timeout.

## Painel web não carrega dados

---

### Causas comuns

- `NEXT_PUBLIC_API_BASE_URL` incorreta;
- API fora do ar;
- CORS ou rota errada;
- frontend apontando para ambiente errado.

### Verificar

- abrir a tela `Status/API Docs`;
- conferir o valor de `API Base`;
- testar `GET /health` diretamente na URL configurada.

## Swagger não abre

---

### Possíveis causas

- API ainda não subiu por completo;
- erro de rota reversa via Nginx;
- problema de proxy em produção.

### Validação

- testar localmente `GET /docs`;
- em produção, validar healthcheck local da API;

- revisar configuracoes de Nginx.

## Analytics vazio

---

### Causas comuns

- nenhuma execucao persistida ainda;
- fluxo rodou, mas falhou antes de gerar telemetria;
- filtros muito restritivos.

### O que fazer

- executar um fluxo simples;
- consultar `GET /api/v1/history`;
- remover filtros de `flow_type`, `model`, `from` e `to`.

## Historico sem detalhes esperados

---

### Possiveis causas

- cache hit com pouca execucao nova;
- integracao sem telemetria retornada pelo provider;
- falha antes da gravacao de determinados steps.

### Validacao

- abrir `GET /api/v1/history/:id`;
- comparar execucao original e execucao cache hit;
- conferir steps gravados.

## Docker sobe, mas a web não abre

---

### Verificar

- `WEB_PORT_MAP`
- logs do container `web`
- build do Next.js
- disponibilidade da API de backend

## Deploy na VPS não conclui

---

### Verificações úteis

1. revisar o workflow do GitHub Actions;
2. validar secrets de SSH;
3. revisar `scripts/deploy.sh`;
4. revisar logs dos containers;
5. validar `nginx -t` no host;
6. testar healthchecks locais `127.0.0.1:3020` e `127.0.0.1:3021`.

## Retry parece aumentar muito a latência

---

### Causa provável

Valores altos de tentativas e delay base podem elevar o tempo total de espera em falhas externas.

### O que revisar

- `EXTERNAL_RETRY_ATTEMPTS`
- `EXTERNAL_RETRY_BASE_DELAY_MS`

## FAQ rapido

---

### Preciso de GitHub e Jira para todos os fluxos?

Nao. Eles sao relevantes principalmente para `review/pull-request` e `tests/pull-request`.

### Posso rodar sem LangSmith?

Sim. O tracing e opcional.

### Posso rodar sem webhook?

Sim. O callback e opcional.

### Posso rodar sem painel?

Sim. A API funciona independentemente da interface web.

### O case depende de Ollama?

Nao. O provedor padrao atual e OpenRouter.

## Relacao com o proximo capitulo

---

Depois deste capitulo, o `15-glossario.md` ajuda a consolidar os principais termos tecnicos usados ao longo da documentacao.

# Glossario

---

## Objetivo deste capítulo

---

Este glossario resume os termos técnicos e de domínio mais usados ao longo da documentação do projeto.

## Termos principais

---

### Agent

Componente orientado a LLM que recebe contexto, segue um prompt e retorna uma saída estruturada.

### Aggregator

Agente responsável por consolidar as saídas de outras etapas ou especialistas em uma resposta final única.

### Analytics

Camada que agrega execuções e telemetria para gerar visão operacional por dia, por fluxo e por modelo.

### Batch

Fluxo que executa vários itens de **review**, **compliance**, **document** e **tests** em sequência em uma única request.

### Cache hit

Situação em que uma execução reutiliza o resultado de outra chamada anterior com o mesmo hash de payload.

### Code standards

Padroes de projeto usados no review de Pull Request para avaliar aderencia do diff a regras ou convencoes locais.

## **Compliance**

Fluxo que compara tarefa e codigo para avaliar aderencia funcional.

## **Controller**

Camada HTTP que valida input, chama o service correto e devolve a resposta da API.

## **Deterministic tools**

Heurísticas e análises previsíveis que rodam sem LLM e complementam os fluxos.

## **Document**

Fluxo que gera documentacao tecnica ou operacional a partir de codigo.

## **Engine**

Camada operacional do fluxo. Coordena cache, persistencia, telemetria, webhook e chamada do grafo principal.

## **Execution**

Registro persistido de uma execucao de fluxo.

## **ExecutionStep**

Registro persistido de uma etapa intermediaria de execucao.

## **ExecutionTelemetry**

Registro persistido de tokens, custos, provider e metadados de uso do modelo.

## **Flow**

Capacidade funcional da API, como **review**, **compliance**, **document**, **tests** ou **batch**.

## **FlowType**

Tipo logico de fluxo usado em governanca de prompts e persistencia.

## **Graph**

Orquestracao de nodes e transicoes que define como um fluxo de IA se comporta.

## **Healthcheck**

Endpoint simples usado para validar se a API esta operacional.

## **History**

Camada de consulta das execucoes persistidas, incluindo steps e telemetria.

## **LangChain**

Biblioteca usada para structured output, mensagens e integracao com o modelo.

## **LangGraph**

Biblioteca usada para modelar os fluxos como grafos explicitos de execucao.

## **LangSmith**

Ferramenta opcional de tracing para fluxos de IA.

## **Model catalog**

Catalogo persistido de modelos permitidos pela API.

## **Model override**

Substituicao pontual do modelo padrao global em uma request especifica.

## **OpenAPI**

Especificacao de contrato HTTP publicada pela API e usada pelo Swagger UI.

## OpenRouter

Provedor LLM utilizado como gateway para modelos e telemetria associada.

## Panel / Painel admin

Interface web administrativa em `apps/web` que consome a API.

## PR review

Fluxo de review aplicado a um Pull Request do GitHub, com Jira opcional.

## PR tests

Fluxo que gera testes unitarios a partir das alteracoes de um Pull Request.

## Prompt block

Bloco individual de prompt dentro de uma versao de fluxo, como `agent`, `security` ou `aggregator`.

## Prompt version

Versao persistida de um conjunto de prompts de um fluxo.

## Prompt override

Uso do campo `prompt_version` para forçar uma versao especifica em uma request.

## Repository

Camada responsavel por ler ou gravar dados persistidos.

## Retry com backoff

Estrategia de repeticao de chamadas externas em falhas transitorias, com atraso progressivo entre tentativas.

## Review



Fluxo de revisao automatizada de codigo, com especialistas por criterio.

## Service

Camada de aplicacao que intermedia controller e engine, expondo o fluxo para a interface HTTP.

## SSE

Server-Sent Events. Mecanismo usado para stream do endpoint `/api/v1/review/stream`.

## Step recorder

Interface usada pelos grafos para registrar etapas no historico quando houver persistencia disponivel.

## Structured output

Resposta do LLM validada contra schema estruturado, e nao texto livre sem contrato.

## Supported language

Conjunto de linguagens aceitas pelos fluxos de codigo:

- `typescript`
- `javascript`
- `python`
- `php`

## Tests

Fluxo que gera testes unitarios a partir de codigo informado.

## Webhook callback

POST opcional enviado pela API ao final de certos fluxos, sem substituir a resposta HTTP principal.